

Departamento de Engenharia Informática
**SWitCH Cyber - Pós-Graduação em Segurança de Informação,
Cibersegurança e Privacidade**

Arquitetura, Desenho e Desenvolvimento de Software Seguro (ADDSS)
Professor Paulo Baltarejo Sousa

Ano letivo 2025/2026

M2: Project

Grupo 3:

Sara Bastos, 1250420

Marta Ferreira, 1201845

Carlos Silva, 1960110

José Rodrigues, 1940368

Index

1	Application Overview	4
2	Threat Modelling	4
2.1	Step 1: Scope the Work	4
2.2	Step 2: Determine Threats	13
2.3	Step 3: Determine Countermeasures and Mitigation.....	15
2.4	Step 4: Assess the Work	16
3	Static Analysis	17
3.1	Automated Static Analysis.....	17
3.2	Manual Static Analysis - File by File.....	18
4	Dynamic Analysis with OWASP ZAP.....	27
4.1	Testing with Postman	27
4.2	Vulnerabilities Found by OWASP ZAP	28
5	Detailed Vulnerability Table	29
5.1	Severity Legend	29
5.2	Main Vulnerability and Mitigation Tables	29
5.3	ASVS Level 2 Gap Analysis	34
6	Mitigation Strategies and Security Implementation	36
6.1	Missing Content Security Policy (CSP) header	36
6.2	Missing Anti-clickjacking header	37
6.3	Application Error disclosure	37
6.4	In-page banner/Tomcat Version Leak.....	38
6.5	Timestamp disclosure	38
6.6	X-Content-Type-Options missing	39
6.7	Missing Content-Type header	39
6.8	H2 Console exposed.....	40
6.9	Additional failures.....	40
7	Conclusion	41
8	References.....	42
9	Annex 1 - TutorialWS_DFD.....	43
10	Annex 2 - SpotBugs Report.....	46
11	Annex 3 - ZAP Report.....	51

Table 1 - Key metadata	4
Table 2 - External Dependencies	5
Table 3 - Entry Points - Tutorials (/api/tutorials)	5
Table 4 - Entry Points - Comments (/api/comments and /api/tutorials/{id}/comments)	6
Table 5 - Entry Points - Tags (/api/tags and /api/tutorials/{id}/tags)	7
Table 6 - Exit Points	8
Table 7 - Assets	9
Table 8 - Trust Levels	9
Table 9 - STRIDE threat list	13
Table 10 - DREAD risk model	15
Table 11 - STRIDE threats	15
Table 12 - Automated Static Analysis	17
Table 13 - Postman tests	27
Table 14 - OWASP ZAP vulnerabilities	28
Table 15 - ASVS: Severity legend	29
Table 17- ASVS: Automatic Static Analysis	30
Table 17 - ASVS: Threat Analysis - Critical Severity	31
Table 18 - ASVS: Threat Analysis: High Severity	31
Table 19 - ASVS: Threat Analysis - Medium Severity	32
Table 20 - ASVS: Threat Analysis - Low Severity	33
Table 21 - ASVS: Dynamic Analysis - OWASP ZAP	33
Table 22 - ASVS Level 2 Gap Analysis	34
Figure 1 - DFD Context	10
Figure 2 - DFD Nivel 1	10
Figure 3 - DFD Tutorials Management	10
Figure 4 - DFD Comments Management	10
Figure 5 - DFD Tags Management	10
Figure 6 - Recommended Implementation to CSP	37
Figure 7 - Current code of ErrorMessage.java	38
Figure 8 - Recommended implementation of ErrorMessage.java	38
Figure 9 -Updated ControllerExceptionHandler.java	39
Figure 10 - Recommended pattern to ResponseEntity	39
Figure 11 - Alternative using @ResponseStatus	39
Figure 12 - Current code used by different entities	40
Figure 13 - Current code of Tutorial class	40
Figure 14 - Improved code of Tutorial class	41
Figure 15 - Database Schema	41
Figure 16 - Current Code of Tutorial.removeTag	41
Figure 17 - Recommended code to Tutorial.removeTag	41

1 Application Overview

The TutorialWS app is a Spring Boot REST API, built using Java, that handles tutorials, comments, and tags through different HTTP endpoints. It also connects to an external service to validate NIF (taxpayer numbers), using a reactive WebClient (WebFlux), this service is available at <http://pbs.dei.isep.ipp.pt:8080/nifs-app>.

For storing data, the app uses an H2 database saved as a file, keeping all the data locally on disk instead of using a separate database server. Depending on how the app is used, this can affect things like security, portability, and how the data is managed.

All the API endpoints are available through HTTP on port 8080, without encryption, because of that, any data sent to or from the app is not protected, which could be risky if used over an unsafe or public network.

2 Threat Modelling

This document follows the OWASP Threat Modelling Community framework, which defines four sequential steps (Scope the Work, Determine Threats, Determine Countermeasures and Mitigation, Assess the Work):

2.1 Step 1: Scope the Work

This step defines the scope of the threat model by documenting the application context, identifying all system components, data flows, external dependencies, and trust boundaries. The outputs are used as the foundation for threat identification in Step 2.

2.1.1 Threat Model Information

The following table captures the key metadata about this threat model engagement:

Table 1 - Key metadata

Field	Value
Application Name	TutorialWS REST API
Version Under Review	v1.0 (Spring Boot 2.6.7)
Author(s)	
Date	2026
Application Owner	ISEP - Departamento de Engenharia Informática
Technology	Java 11, Spring Boot 2.6.7, H2 File DB, Spring WebFlux
Methodology	OWASP Threat Modeling- STRIDE + DREAD
Document Status	Final
Reviewed By	

2.1.2 External Dependencies

External dependencies are components outside the application's direct control that are integrated via configuration or code. Each of the following represents a potential attack surface.

Table 2 - External Dependencies

Description	Location
H2 in-memory/file database (v2.1.210)-included via Spring Boot starter	Embedded in JVM process
NIF validation external service- pbs.dei.isep.ipp.pt:8080/nifs-app	Remote HTTP service (no SLA)
Spring Boot 2.6.7 (parent POM)- includes all Spring dependencies	Maven Central / pom.xml
Spring WebFlux / Project Reactor- used for async WebClient	Included via spring-boot-starter-webflux
H2 Console web UI- accessible at /h2-ui (enabled in application.properties)	Embedded servlet container

2.1.3 Entry Points

Entry points are all interfaces through which data and commands can enter the system. Each entry point is a potential attack vector and must be protected. The table shown below maps all HTTP endpoints identified in the TutorialWS API:

The TutorialController handles CRUD operations on tutorials. It accepts CORS requests from <http://localhost:8080>.

Table 3 - Entry Points - Tutorials (/api/tutorials)

Method	URL	Auth Required	Trust Level	Description	Comments
GET	/api/tutorials	None	Public	List all tutorials. Optional ?title=search.	<i>No credentials checked. Any client can read all data.</i>
GET	/api/tutorials/{id}	None	Public	Get tutorial by ID. 404 if not found.	<i>Exposes individual records without restriction.</i>
GET	/api/tutorials/published	None	Public	List published=true tutorials only.	<i>Read-only, filtered view: lowest risk endpoint.</i>
POST	/api/tutorials	None	Internal	Create tutorial. published forced to true.	<i>Write operation with no auth: should be protected in production.</i>

PUT	/api/tutorials/{id}	None	Sensitive	Update title, description, published.	<i>Modifies existing records. No auth or ownership check.</i>
DELETE	/api/tutorials/{id}	None	Sensitive	Delete tutorial by ID.	Permanent deletion with no confirmation or auth.
DELETE	/api/tutorials	None	Sensitive	Delete ALL tutorials.	<i>Destroys all data: critical risk with no safeguard.</i>

Trust Level: **Public** = no restrictions **Internal** = should be network-restricted in production **Sensitive** = destructive / high-impact operation | **Auth:** **None** = no authentication in current code **NIF Validated** = external NIF check before write

The CommentController manages comments associated with tutorials. It supports CORS from http://localhost:8080. Important note: When creating a comment, the nif field is validated externally before being saved.

Table 4 - Entry Points - Comments (/api/comments and /api/tutorials/{id}/comments)

Method	URL	Auth Required	Trust Level	Description	Comments
GET	/api/tutorials/{tutorialId}/comments	None	Public	List all comments for a tutorial.	<i>404 if tutorial not found. No auth required to read comments.</i>
GET	/api/comments/{id}	None	Public	Get a comment by ID.	<i>Direct comment lookup :no ownership or session check.</i>
POST	/api/tutorials/{tutorialId}/comments	NIF Validated	Internal	Create comment. NIF validated externally before saving.	<i>Only write endpoint with any validation. 204 if NIF rejected by external service.</i>
PUT	/api/comments/{id}	None	Sensitive	Update comment content.	<i>Any caller can overwrite any comment :no ownership check.</i>
DELETE	/api/comments/{id}	None	Sensitive	Delete comment by ID.	<i>Permanent delete with no auth or confirmation.</i>
DELETE	/api/tutorials/{tutorialId}/comments	None	Sensitive	Delete all comments of a tutorial.	<i>Bulk delete :all comments wiped with one</i>

					<i>unauthenticated call.</i>
--	--	--	--	--	------------------------------

Trust Level: **Public** = no restrictions **Internal** = should be network-restricted in production **Sensitive** = destructive / high-impact operation | **Auth:** **None** = no authentication in current code **NIF Validated** = external NIF check before write

The TagController manages tags and their associations with tutorials. It supports CORS from <http://localhost:8081> (a different port than the other controllers).

Table 5 - Entry Points - Tags (/api/tags and /api/tutorials/{id}/tags)

Method	URL	Auth Required	Trust Level	Description	Comments
GET	/api/tags	None	Public	List all tags.	<i>Unrestricted read: returns all tags in the system.</i>
GET	/api/tags/{id}	None	Public	Get tag by ID.	<i>404 if not found. No auth required.</i>
GET	/api/tutorials/{tutorialId}/tags	None	Public	List tags of a specific tutorial.	<i>404 if tutorial not found.</i>
GET	/api/tags/{tagId}/tutorials	None	Public	List tutorials that have a specific tag.	<i>Cross-reference query: no restriction.</i>
POST	/api/tutorials/{tutorialId}/tags	None	Internal	Add/create tag on tutorial. id=0 creates new.	<i>No auth. Unauthenticated caller can associate any tag with any tutorial.</i>
PUT	/api/tags/{id}	None	Sensitive	Update tag name.	<i>Affects all tutorials linked to this tag: no auth or audit trail.</i>
DELETE	/api/tutorials/{tutorialId}/tags/{tagId}	None	Internal	Remove tag association (tag not deleted).	<i>Unlinks only: does not cascade to other associations.</i>
DELETE	/api/tags/{id}	None	Sensitive	Delete tag by ID.	<i>Removes tag + all tutorial_tags rows for this tag. No confirmation.</i>

Trust Level: **Public** = no restrictions **Internal** = should be network-restricted in production **Sensitive** = destructive / high-impact operation | **Auth:** **None** = no authentication in current code **NIF Validated** = external NIF check before write

2.1.4 Exit Points

The application has four distinct exit points: channels through which data or signals leave the process boundary. Unlike the entry points in section 2.1.3 (inbound HTTP), exit points are outbound interactions initiated by the application itself.

Table 6 - Exit Points

Type	Destination	Auth Required	Trust Level	Description	Comments
HTTP GET	pbs.dei.isep.ipp.pt:8080 /nifs-app/nifs/{id}	None	Sensitive	Outbound call to external NIF validation service. Triggered on every POST /comments. Sends NIF number as path variable.	<i>No credentials: unauthenticated call to a remote server. If the service is compromised or unreachable, comment creation silently returns 204.</i>
DB Write	H2 file tutorialsdh2.mv.db	SA / no pwd	Internal	JPA/Hibernate writes to local H2 file on every save() and delete() call across all three repositories.	<i>File stored in the working directory. No password protection. Direct file access would expose all application data.</i>
DB Read	H2 file tutorialsdh2.mv.db	SA / no pwd	Internal	JPA/Hibernate reads on all GET endpoints via findAll(), findById(), findBy* derived queries.	<i>Same file-based H2 as writes. All read queries are unfiltered: no row-level security or tenant isolation.</i>
H2 Console	HTTP :8080 /h2-ui (browser)	None	Sensitive	Spring Boot serves the H2 web admin console at /h2-ui. Any browser with network access can connect and run arbitrary SQL.	<i>Enabled in application.properties line 19. No authentication configured. Must be disabled before any production deployment.</i>
STDOUT Log	Process stdout (console / container log)	N/A	Public	System.out.println() in CommentController logs WebClient error messages. SLF4J Logger declared in WebFluxConfig (never used).	<i>Errors from NIF service calls are printed to stdout. May leak external service URLs or stack traces into container logs.</i>

Trust Level: **Public** = informational only **Internal** = should be isolated from public network **Sensitive** = high risk: requires auth or removal before production

2.1.5 Assets

These are the resources and data that the application manages and that require protection:

Table 7 - Assets

Asset Name	Description	Risk if Compromised
Tutorial data	All tutorials (title, description, published state, tags)	High - data loss/manipulation
Comment data + NIFs	Comments linked to tutorials; NIF (taxpayer numbers) validated via external service	Critical - PII exposure
Tag data	Tags associated with tutorials	Medium - data manipulation
H2 database file (tutorialsdh.h2)	Local file containing all application data; accessible via H2 Console	Critical - full data breach
Application source code	Spring Boot application; hardcoded URLs and config	High - secrets/logic exposure
Application.properties	Config file with DB credentials (blank password), H2 console path	Critical - credential exposure
Application availability	REST API uptime; no rate limiting means easy DoS	High - service disruption

2.1.6 Trust Levels

The trust levels represent the access rights that the application grants to external entities, they are used to define the trust boundaries and are referenced in the threat analysis:

Table 8 - Trust Levels

Name	Description
Anonymous User	Any HTTP client (browser, curl, Postman) with network access to port 8080. Has FULL access to all API endpoints due to absence of authentication.
API Consumer	Application or user making REST calls. Currently no distinction from TL-01 - no authentication enforced.
Database Administrator	Any user who accesses /h2-ui (H2 Console). Currently equivalent to TL-01 due to missing protection.
External NIF Service	Remote service at pbs.dei.isep.ipp.pt:8080. No authentication, no certificate validation, no timeout configured.
Application Process (JVM)	Spring Boot JVM process on the host. Has full read/write access to tutorialsdh.h2 database file.

System Administrator	Intended privileged role - NOT IMPLEMENTED. Should have access to DELETE all, manage H2 console.
----------------------	--

2.1.7 Data Flow Diagrams

The Data Flow Diagrams (DFD) below illustrates the interactions between system components and highlights trust boundary crossings. Components within the dashed boundary are part of the application deployment; components outside represent external entities.

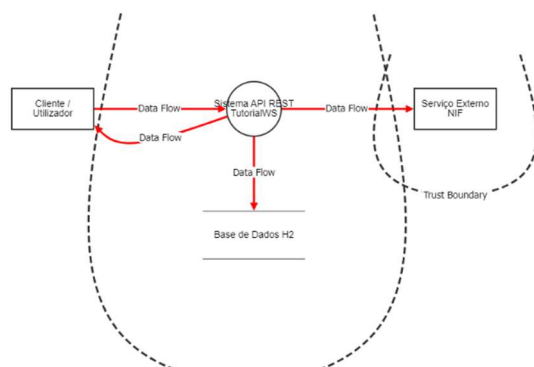


Figure 1 - DFD Context

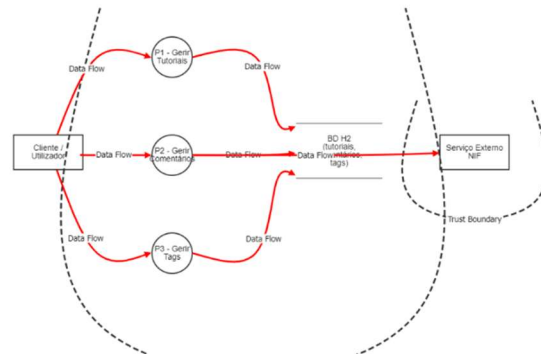


Figure 2 - DFD Nivel 1

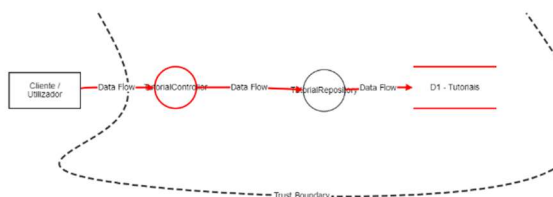


Figure 3 - DFD Tutorials Management

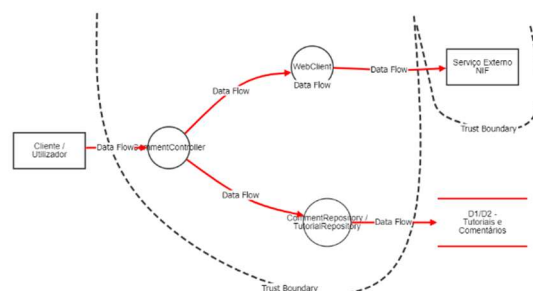


Figure 4 - DFD Comments Management

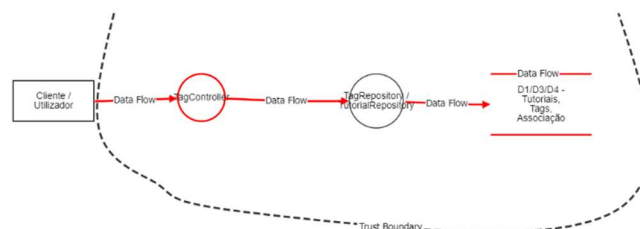


Figure 5 - DFD Tags Management

2.1.8 Application Architecture Detail

This section goes through the application's source files and explains what each part does, especially anything related to security.

The application is built using the Model-View-Controller (MVC) pattern and is organized into five main layers:

- a) Controllers: receive HTTP requests and return JSON responses
 - TutorialController
 - CommentController
 - TagController
- b) Model: data entities persisted in the database
 - Tutorial
 - Comment
 - Tag
- c) Repository: database access via JPA
 - TutorialRepository
 - CommentRepository
 - TagRepository
- d) H2 Database: local file tutorialsdb.h2
- e) External dependency: CommentController contacts an external service at [url = http://pbs.dei.isep.ipp.pt:8080/nifs-app](http://pbs.dei.isep.ipp.pt:8080/nifs-app), to validate a NIF before creating comments.

Overview - Functional Explanation

This application is composed by the following parts:

a) Tutorial.java - Model

This class represents a tutorial. Spring/JPA uses it to automatically create and map the tutorials table in the database, where each object of this class matches a row in that table.

It has four main fields:

- id (automatically generated)
- title
- description
- published

It also has a relationship with Tag, since a tutorial can have multiple tags, and a tag can belong to multiple tutorials, therefore, a junction table tutorial_tags is automatically created in the database for this purpose.

The empty constructor is required because Spring needs it to convert received JSON into a Java object.

b) Controller - TutorialController.java

This is the central file of the project, here all HTTP routes available in the API are defined with the prefix */api*. Each method corresponds to an operation:

- ***getAllTutorials***
 - responds to *GET /api/tutorials* and returns all tutorials.
 - If a (*?title=value*) parameter is passed in the URL, it filters by title. If none exist, it returns *204 No Content*.
- ***getTutorialById***
 - responds to *GET /api/tutorials/{id}* and returns a specific tutorial.
 - If it does not exist, it throws an exception that the handler converts to a *404 Not Found*.
- ***createTutorial***
 - responds to *POST /api/tutorials*.
 - It receives the JSON body, converts it to a tutorial object, saves it to the database and returns the created tutorial with the generated id, with code *201 Created*.
- ***updateTutorial***
 - responds to *PUT /api/tutorials/{id}*.
 - It fetches the existing tutorial, updates the fields with the values received in the body, and saves it again.
- ***deleteTutorial* and *deleteAllTutorials***
 - responds to *DELETE* and deletes one or all tutorials respectively.
- ***findByPublished***
 - responds to *GET /api/tutorials/published* and returns only tutorials with *published = true*.

c) Error Handling - ControllerExceptionHandler.java

This file acts as a global error interceptor, whenever any controller throws an exception, this file catches it and returns a formatted JSON response with the error code, date, message and the URI where it occurred.

It handles two cases:

- If the tutorial is not found (*ResourceNotFoundException*) it returns *404 Not Found*
- For any other unexpected exception, it returns *500 Internal Server Error*.

d) WebClient Configuration - WebFluxConfig.java

This sets up a *WebClient*, which is basically an HTTP client used by the application to call external APIs, in this case ISEP API. It can be registered as a *@Bean* and then reused in other parts of the app using *@Autowired* or constructor injection.

According to Spring's documentation, *WebClient* is a reactive (non-blocking) HTTP client. Even so, it does not have to be tied to the controller layer, many Spring MVC apps use *WebFlux* just to take advantage of *WebClient*.

The issue here comes from implementing *WebFluxConfigurer*, which ends up enabling the reactive stack in Spring. The use of *WebClient* in a traditional Spring MVC application requires careful configuration, particularly regarding reactive and blocking interactions.

e) Configuration - application.properties

This defines how the app connects to the H2 database, stored locally in *tutorialsdb.h2* file. The *ddl-auto=update* setting makes Spring automatically create or update the database tables when the app starts, based on the *@Entity* classes.

It also turns on the H2 web console, which can be accessed at <http://localhost:8080/h2-ui> to view and check the database directly from the browser.

2.2 Step 2: Determine Threats

Using the STRIDE methodology, each component identified in Step 1 is analysed for potential threats, which are classified into six categories:

- Spoofing
- Tampering
- Repudiation
- Information Disclosure
- Denial of Service
- Elevation of Privilege

For each threat, the affected component, description, impact, and initial recommendation are indicated.

2.2.1 STRIDE Threat List

The table below maps all the identified threats to their STRIDE category, affected component, and impact.

Table 9 - STRIDE threat list

Component	STRIDE Category	Threat Description	Impact
REST API	S - Spoofing	Any user can impersonate another. There is no authentication on any endpoint.	Critical
DELETE /tutorials	T - Tampering	Any unauthenticated client can delete all tutorials with a single DELETE request.	Critical
POST /comments	T - Tampering	The "content" field of a comment has no size limit (@Lob), allowing injection of large data.	Medium
REST API	R - Repudiation	No operation logging exists. Impossible to audit who created, modified, or deleted resources.	Medium
H2 DB / /h2-ui	I - Info Disclosure	The H2-UI console is publicly exposed at /h2-ui without authentication. Full database access.	Critical

Component	STRIDE Category	Threat Description	Impact
application.properties	I - Info Disclosure	Empty database password (spring.datasource.password=)Credentials in plain text in the configuration file.	High
External NIF service	I - Info Disclosure	The external service URL is hardcoded in WebFluxConfig.java. If the service fails, it returns null without throwing the correct error.	Medium
REST API	D - Denial of Service	No rate limiting. An attacker can send thousands of requests and exhaust application resources.	High
CORS - TutorialCtrl	E - Elevation of Priv.	@CrossOrigin(origins="http://localhost:8080") only allows local origin, but TagController uses port 8081 - inconsistency.	Low
PUT /comments/{id}	E - Elevation of Priv.	Any user can edit another user's comment. There is no ownership verification.	High
All REST endpoints (HTTP)	I - Info Disclosure	All client-server communication uses plain HTTP. NIFs, content and metadata are transmitted without encryption, exposing them to interception (MITM).	High
CommentController (Optional.of(nif))	D - Denial of Service	When the NIF Service is unavailable, onErrorReturn(null) returns null. The subsequent Optional.of(nif) call throws an unhandled NullPointerException, crashing the request handler.	High
WebFluxConfig.java (WebClient)	D - Denial of Service	No responseTimeout, connectionTimeout or readTimeout is configured on the WebClient. If the NIF Service becomes unresponsive, requests block indefinitely and exhaust the thread pool.	Medium
application.properties (ddl-auto)	T - Tampering	spring.jpa.hibernate.ddl-auto=update allows Hibernate to automatically alter the database schema on every startup, which can cause data loss or schema inconsistency in production.	Medium

2.2.2 Ranking of Threats - Qualitative Risk Analysis (DREAD)

The DREAD risk model is used to evaluate threats based on probability, impact, and how easy they are to exploit, helping to define a risk level.

This prioritization helps when deciding what countermeasures to apply in Step 3:

Table 10 - DREAD risk model

Threat	Probability	Impact	Ease of Exploitation	Priority
H2-UI exposed without authentication	High	Critical	Very easy	Critical
No authentication on all endpoints	High	Critical	Very easy	Critical
DELETE /api/tutorials unprotected	High	Critical	Very easy	Critical
Empty DB password in config	Medium	High	Easy	High
No rate limiting	High	High	Easy	High
Editing other users' comments	High	High	Easy	High
No HTTPS / TLS on endpoints	High	High	Easy	High
NPE via Optional.of(null)	High	High	Easy	High
No input validation	Medium	Medium	Moderate	Medium
Hardcoded external service URL	Low	Medium	Difficult	Medium
WebClient without timeout	Medium	Medium	Moderate	Medium
DDL auto-update in production	Low	Medium	Difficult	Medium
No logging / auditing	Medium	Medium	Moderate	Medium
Inconsistent CORS policy	Low	Low	Difficult	Low

2.3 Step 3: Determine Countermeasures and Mitigation

For each threat identified in Step 2, a specific countermeasure is defined to reduce or remove the associated risk. These countermeasures are linked to OWASP ASVS v4 requirements so they can be properly checked and validated.

2.3.1 STRIDE Threat & Mitigation Techniques

The following table maps each identified threat to its STRIDE category, recommended countermeasure and the corresponding ASVS requirement:

Table 11 - STRIDE threats

Threat	STRIDE	Countermeasure	ASVS Ref.
No authentication on any endpoint	Spoofing	Add Spring Security to pom.xml. Configure JWT/OAuth2. Annotate all endpoints with @PreAuthorize.	V4.1.1
H2-UI console publicly exposed	Information Disclosure	Set spring.h2. console.enabled=false in production profile.	V14.2.2
DELETE /api/tutorials unprotected	Tampering	Require ADMIN role via @PreAuthorize. Add confirmation token before bulk delete.	V4.1.3

Empty DB password in config	Information Disclosure	Externalise to env var: <code>spring.datasource.password=\${DB_PASSWORD}</code> . Never commit to VCS.	V2.10.1
No rate limiting	DoS	Integrate Bucket4j or Spring Cloud Gateway rate-limiting filter per IP.	V11.1.2
Any user can edit any comment	Elev. Priv.	Verify comment ownership before update/delete. <code>@PreAuthorize</code> with principal check.	V4.2.1
No input validation (<code>@Valid</code> missing)	Tampering	Add <code>spring-boot-starter-validation</code> . Use <code>@Valid</code> , <code>@NotBlank</code> , <code>@Size</code> on all DTOs.	V5.1.3
WebClient without TLS	Information Disclosure	Change URL to <code>https://</code> . Configure <code>SSLContext</code> in <code>ReactorClientHttpConnector</code> .	V9.1.1
Hardcoded NIF service URL	Information Disclosure	Move URL to <code>application.properties</code> as <code>nif.service.url</code> . Inject with <code>@Value</code> .	V2.10.1
No logging / auditing	Repudiation	Implement <code>SLF4J/Logback</code> . Log all write operations with user, timestamp, resource.	V7.3.1
Inconsistent CORS policy	Elev. Priv.	Centralise CORS in <code>WebSecurityConfig</code> . Remove per-controller <code>@CrossOrigin</code> .	V14.5.3
No HTTPS / TLS on endpoints	Information Disclosure	Configure <code>SSL/TLS</code> in Spring Boot (<code>server.ssl.*</code>). Redirect all HTTP to HTTPS.	V9.1.1
<code>NullPointerException</code> via <code>Optional.of(null)</code>	DoS	Replace <code>Optional.of(nif)</code> with <code>Optional.ofNullable(nif)</code> . Add <code>Resilience4j</code> circuit breaker.	V11.1.2
WebClient without timeout	DoS	Configure <code>responseTimeout</code> and <code>connectionTimeout</code> on <code>HttpClient</code> .	V11.1.2
DDL auto-update in production	Tampering	Change to <code>ddl-auto=validate</code> or <code>none</code> . Use <code>Flyway</code> or <code>Liquibase</code> for migrations.	V14.2.2

2.4 Step 4: Assess the Work

This step checks how complete and good the threat model is, making sure all important threats were found and properly handled. The review is done using three different methods: automated static analysis, manual code review and dynamic testing with OWASP ZAP. The results are shown in Sections 3 and 4.

The main findings were:

- 14 threats identified across 6 STRIDE categories
- 3 are Critical, 5 High, 5 Medium and 1 Low priority
- None of the threats currently have countermeasures implemented
- Static analysis supports the identification of several threats using automated tools and manual review
- Dynamic analysis confirms STRIDE results using OWASP ZAP
- ASVS Level 2 compliance: 18 missing, 3 partial, 1 OK (full gap analysis in Section 5.3)

3 Static Analysis

The static analysis consists in checking the source code without executing the application and is divided into automated analysis (using tools) and manual analysis.

3.1 Automated Static Analysis

For the automated static analysis, the SpotBugs plugin for IntelliJ IDEA was used, this is an open-source tool that scans Java bytecode (compiled .class files) and looks for patterns that might indicate bugs or security weaknesses.

This plugin makes the process easier since it runs directly inside the IDE, in the development environment and shows the results right next to the code.

In this case, the analysis found 9 warnings, all with medium priority, divided into two categories as shown in the table below:

Table 12 - Automated Static Analysis

Category	Location	Description	Priority
Correctness	Tutorial.java:72 - removeTag(long)	Possible null pointer dereference of tag. Variable is known null at line 71 and dereferenced at line 72.	Medium
Malicious Code	ErrorMessage.java:23 - getTimestamp()	May expose internal representation by returning a direct reference to the mutable Date field timestamp.	Medium
Malicious Code	Comment.java:40 - getTutorial()	May expose internal representation by returning a direct reference to the mutable Tutorial field tutorial.	Medium
Malicious Code	Tag.java:39 - getTutorials()	May expose internal representation by returning a direct reference to the mutable Set field tutorials.	Medium
Malicious Code	Tutorial.java:76 - getTags()	May expose internal representation by returning a direct reference to the mutable Set field tags.	Medium
Malicious Code	ErrorMessage.java:13 - constructor	May expose internal representation by storing an externally mutable Date object directly into the timestamp field.	Medium
Malicious Code	Comment.java:44 - setTutorial(Tutorial)	May expose internal representation by storing an externally mutable Tutorial object directly into the tutorial field.	Medium
Malicious Code	Tag.java:42 - setTutorials(Set)	May expose internal representation by storing an externally mutable Set object directly into the tutorials field.	Medium
Malicious Code	Tutorial.java:80 - setTags(Set)	May expose internal representation by storing an externally mutable Set object directly into the tags field.	Medium

The *Correctness* warning is about a potential crash in the *removeTag* method in *Tutorial.java*, the code uses a variable that might be null, which could lead to a *NullPointerException* at runtime.

The other 8 warnings are all about the same issue, some getters and setters in the model classes return or store objects like *Date* or *Set* directly, instead of returning a copy. This means that if those objects are changed outside the class, the internal data also gets changed, which is a bad practice from a security point of view.

3.2 Manual Static Analysis - File by File

Each project file of the *TutorialWS* REST API Java script was analysed individually:

a) WebFluxConfig.java

TutorialWS/src/main/java/com.example.tutorialws/config/WebFluxConfig.java

1st Identified Problem - URL with http:// (no TLS)

The URL <http://pbs.dei.isep.ipp.pt:8080/nifs-app> uses plain HTTP, this means that communication between the application and the external NIF service is in clear text and an attacker on the network can intercept and read the NIFs being transmitted.

File	WebFluxConfig.java, line 26
Problem	Communication with external service without TLS
OWASP Category	A04:2025 - Cryptographic Failures

2nd Identified Problem - Missing timeout in WebClient

In the line 29, *ConnectionProvider.newConnection()* is used, which creates a new TCP connection for each request, inefficient and without limits.

File	WebFluxConfig.java, line 29
Problem	WebClient without timeout or connection pool
OWASP Category	A02:2025 - Security Misconfiguration

b) TutorialController.java

src/main/java/com.example.tutorialws/controller/TutorialController.java

3rd Problem - @CrossOrigin restricted to localhost

The *@CrossOrigin* (origins = "<http://localhost:8080>") only allows requests from the local machine, in development this appears safe, but there are two issues:

- In production, if someone copies this code, they may forget to update it and end up with a misconfigured CORS policy
- TagController has *@CrossOrigin* (origins = "<http://localhost:8081>"), different origins in different controllers, which is inconsistent

File	TutorialController.java, line 23
Problem	CORS configured inconsistently across controllers
OWASP Category	A02:2025 - Security Misconfiguration

4th Problem - @RequestBody without @Valid

In the *createTutorial* method, the parameter receives user data directly without any validation.

File	TutorialController.java, method createTutorial
Problem	Absence of input validation (Bean Validation)
OWASP Category	A05:2025 - Injection / A06:2025 - Insecure Design

5th Problem - deleteAllTutorials() without protection

There is a “DELETE /api/tutorials endpoint” that calls “tutorialRepository.deleteAll()”, it deletes the entire database with a single request, without authentication, confirmation or an audit trail.

File	TutorialController.java, method deleteAllTutorials
Problem	Destructive mass operation without authentication or authorisation
OWASP Category	A01:2025 - Broken Access Control

c) CommentController.java

src/main/java/com.example.tutorialws/controller/CommentController.java

6th Identified Problem - No Authentication on Any Endpoint

All routes in *TutorialController* are open source to everyone without any authentication mechanism. Any client can call every endpoint, even the ones that change or delete data, this happens because the controller at line 26 has no *@PreAuthorize*, Spring Security filter or session checks, so every method is exposed as TL-01 to anonymous users with full access.

File	TutorialController.java, line 26 (class level, all methods)
Problem	All endpoints accessible without authentication or authorisation
OWASP Category	A01:2025 - Broken Access Control

7th Identified Problem - createTutorial Forces published=true Regardless of Input

In the *createTutorial* method at line 56, the code always sets the published argument to true when creating a new tutorial: *new Tutorial(tutorial.getTitle(), tutorial.getDescription(), true)*. It ignores whatever value is sent in the request, which is an insecure design issue, since the user has no control over whether the tutorial should be published or not and it can end up making content public before it is ready.

File	TutorialController.java, method createTutorial (line 56)
Problem	Hardcoded business logic ignores caller-supplied published state
OWASP Category	A06:2025 - Insecure Design

8th Identified Problem - No Pagination on Listing Endpoints

The “getAllTutorials” method at line 35 calls *tutorialRepository.findAll()* without any limits or pagination. If there are lots of records in the database, everything gets loaded into memory and sent back in one go, this can be abused to cause a denial-of-service by repeatedly triggering the endpoint and using up all the server memory.

File	TutorialController.java, method getAllTutorials (line 35)
Problem	Unbounded result set: missing pagination exposes DoS risk
OWASP Category	A06:2025 - Insecure Design / A02:2025 - Security Misconfiguration

9th Problem - URL manipulation via Path Traversal / SSRF

In the “findByld” method, the received id is used directly to construct the URL: *uri("/nifs/" + id)*.

File	CommentController.java, method findByld
Problem	User input used directly to construct a URL without sanitisation
OWASP Category	A01:2025 - Broken Access Control (SSRF)

10th Problem - Silent error handling

Errors are basically ignored, since the app doesn’t log them, it doesn’t tell the user anything and just returns empty responses. This is problematic because:

- Security issues do not show up in logs
- An attacker can try repeatedly without leaving a trace
- Real errors look the same as attacks

File	CommentController.java, method findByld
-------------	---

Problem	Exceptions caught and discarded without logging - absence of auditing
OWASP Category	A09:2025 - Security Logging & Alerting Failures

11th Identified Problem - NIF Validation Causes NullPointerException

In the *findByld* method (line 94), the code uses *onErrorReturn(null)* when calling an external NIF service, this means it can store *null* inside a *Mono*. Then, at line 99, it calls *Optional.of(nif)* with that null value, which always throws a *NullPointerException*, since this error isn't caught it ends up as a 500 response. The problem comes from mixing reactive code with *Optional.of*, which doesn't allow null values.

File	CommentController.java, method findByld (lines 94-99)
Problem	onErrorReturn(null) followed by Optional.of(null) causes unhandled NullPointerException
OWASP Category	A06:2025 - Insecure Design

12th Identified Problem - No Authentication on Comment Endpoints

The *CommentController* at line 25, also has no authentication. Any anonymous user can create, read, update or delete comments on any tutorial. There is no session check, no token validation and no Spring Security filters, which means anyone can post comments using any NIF, change other user's comments or delete them.

File	CommentController.java, line 25 (class level, all methods)
Problem	All comment endpoints publicly accessible without authentication
OWASP Category	A01:2025 - Broken Access Control

13th Identified Problem - PUT /comments/{id} Does Not Verify Comment Ownership

The *updateComment* method (lines 64-68) gets a comment by its id and just replaces the content with whatever comes in the request. There is no check to see if the user owns that comment or has permission to change it, therefore, anyone can edit anyone else's comment just by knowing the id, which makes it easy to pretend to be someone else.

File	CommentController.java, method updateComment (lines 64-68)
Problem	Comment update endpoint allows modification without ownership or authorisation check
OWASP Category	A01:2025 - Broken Access Control

14th Identified Problem - DELETE /comments/{id} Lacks Ownership Verification

The deleteComment method (lines 70-73) deletes a comment by id without checking who is making the request. There's no validation to see if the user owns the comment or is an admin. This means any user can delete any comment just by trying different ids. Since there's no authentication or ownership check, this is a serious risk for data integrity.

File	CommentController.java, method deleteComment (lines 70-73)
Problem	Comment deletion without authentication or ownership verification
OWASP Category	A01:2025 - Broken Access Control

15th Identified Problem - NIF Service Base URL Not Externalised as Configuration

At line 90, the NIF validation path /nifs/ is directly written in the controller. Also, the base URL (<http://pbs.dei.isep.ipp.pt:8080/nifs-app>) is hardcoded in *WebFluxConfig.java*, if this external service changes, the app must be recompiled and deployed again. On top of that, you cannot easily change the URL depending on the environment (like dev, staging, or production) without changing the code, which is not good practice.

File	CommentController.java, method findById (line 90); WebFluxConfig.java (line 26)
Problem	NIF service URL hardcoded: not externalised to configuration
OWASP Category	A02:2025 - Security Misconfiguration

16th Identified Problem - @Lob Field in Comment Entity Without Size Constraint

The *content* field in *Comment.java* at line 12 uses *@Lob* but doesn't define any size limit. There's no *@Column(length=...)* either, this means someone can send a very large comment (even gigabytes) and it will be stored in the database and returned later, which can fill up storage, slow down queries and even be used to cause denial-of-service issues.

File	Comment.java, field content (line 12); triggered via CommentController POST endpoint
Problem	Unbounded @Lob field accepts comment content of arbitrary size
OWASP Category	A06:2025 - Insecure Design

17th Identified Problem - Static Inner Nif Class Has No Field Validation

The *Nif* inner class (lines 104-129) has two fields: *nr* (long) and *name* (String), but there are no validation rules, the *nr* field can be anything like zero, negative numbers or very large values and the *name* field can be null or any size. Since this is used to read data

from an external service, a bad or malicious response could send invalid data and the system would just accept it.

File	CommentController.java, static inner class Nif (lines 104-129)
Problem	NIF response model has no field validation constraints
OWASP Category	A08:2025 - Software and Data Integrity Failures

18th Identified Problem - System.out.println Used for Error Logging

At line 95, errors are printed using *System.out.println*, this is not proper logging and usually is not tracked in production. Important events like failures in the NIF service or network issues will not be recorded in logging systems or monitoring tools, which makes it hard to detect problems or investigate security issues later.

File	CommentController.java, method findById (line 95)
Problem	System.out.println used instead of a structured logger: no audit trail
OWASP Category	A09:2025 - Security Logging & Alerting Failures

d) application.properties

src/main/resources/application.properties

19th Problem - Blank database password

On line 8, *spring.datasource.password=*, has no value, so anyone who knows the JDBC URL can connect to the database without any credentials.

File	application.properties, line 8
Problem	Blank database password
OWASP Category	A07:2025 - Authentication Failures

20th Problem - H2 Console enabled in production

On line 3, the database is set as *jdbc:h2:file:./tutorialsdh.h2*, which means the file is stored in the app's working directory and is easily accessible

On line 7, the username is still set to default *username=sa*, easy to guess and insecure.

On line 20, *spring.h2.console.enabled=true* turns on the H2 web database administration console and available public at */h2-ui*, which lets anyone run SQL queries directly on the database.

On line 24, the *console path=/h2-ui*, is predicable and not protected, since the password is empty, anyone who goes to <http://localhost:8080/h2-ui> can fully access the database from the browser without logging in.

File	application.properties, lines 20 and 24
Problem	H2 administration console enabled and publicly accessible
OWASP Category	A02:2025 - Security Misconfiguration

e) TagController.java

src/main/java/com.example.tutorialws/controller/TagController.java

21st Identified Problem - No authentication on tag endpoints

None of the endpoints in *TagController* require authentication or authorisation, so any user can create, update or delete tags, which affects all tutorials that use those tags.

File	TagController.java, all endpoints
Problem	No authentication or access control on tag operations
OWASP Category	A01:2025 - Broken Access Control

22nd Identified Problem - Inconsistent CORS policy

TagController allows requests from *http://localhost:8081*, while *TutorialController* uses port 8080, this mismatch is inconsistent and can cause problems in production, it might block valid requests or allow ones that should not be allowed.

File	TagController.java, class level annotation
Problem	CORS configured with a different origin than TutorialController
OWASP Category	A02:2025 - Security Misconfiguration

23rd Identified Problem - No input validation on tag fields

The *name* field in tags has no validation, it can be empty, null, or extremely long. This can lead to bad data in the database or even be used for denial-of-service attacks with very large inputs.

File	TagController.java, createTag and updateTag methods
Problem	Absence of input validation on tag name field
OWASP Category	A05:2025 - Injection / A06:2025 - Insecure Design

24th Identified Problem - PUT /tags/{id} Allows Modification Without Ownership Verification

The *updateTag* method (lines 78-82) lets anyone change the name of any tag just by providing its id, there's no check for ownership or permissions. Since tags are shared, changing one tag affects all tutorials that use it and a single request could silently mess up data across the whole system.

File	TagController.java, method updateTag (lines 78-82)
Problem	Tag update endpoint modifies shared data without ownership or permission check
OWASP Category	A01:2025 - Broken Access Control

f) ControllerExceptionHandler.java

src/main/java/com.example.tutorialws/exception/ControllerExceptionHandler.java

25th Identified Problem - Internal exception messages exposed to the client

The global *@ExceptionHandler* returns *ex.getMessage()* directly in the HTTP response body. This can expose internal details like class names, database info or parts of stack traces, which helps attackers understand how the system works.

File	ControllerExceptionHandler.java, handleAllExceptions method
Problem	Internal Java exception messages returned directly in the response
OWASP Category	A10:2025 - Mishandling of Exceptional Conditions / A06:2025 - Insecure Design

26th Identified Problem - Internal request details exposed via getDescription

The *ErrorMessage* class includes *request.getDescription(false)* in the response body, even with the *false* flag hiding the user, this can still expose internal request paths and parameters to the client, which is unnecessary information disclosure.

File	ControllerExceptionHandler.java, ErrorMessage construction
Problem	<i>request.getDescription(false)</i> included in error responses
OWASP Category	A06:2025 - Insecure Design

27th Identified Problem - No Handler for MethodArgumentNotValidException

The *ControllerExceptionHandler* handles general exceptions and but doesn't handle *MethodArgumentNotValidException*. This means when validation fails, Spring uses its

default error response, which may expose field names, rejected values, and validation messages in an uncontrolled way.

File	ControllerExceptionHandler.java (no method handles MethodArgumentNotValidException)
Problem	Missing handler for validation exceptions: raw error details may be exposed
OWASP Category	A02:2025 - Security Misconfiguration

g) pom.xml (Dependencies)

28th Identified Problem - Outdated Spring Boot version with known CVEs

The project is using Spring Boot 2.6.7, which came out in 2022, this has several known CVEs that have been patched in later versions. Using an outdated framework version significantly increases the attack surface, since publicly disclosed vulnerabilities are documented and easy to exploit.

File	pom.xml, spring-boot-starter-parent version
Problem	Spring Boot 2.6.7 - outdated version with known CVEs
OWASP Category	A03:2025 - Software Supply Chain Failures

29th Identified Problem - No security dependency included

The project does not include *spring-boot-starter-security* in its dependencies. This means there is no Spring Security, no filter chain, no authentication manager and no way to add access control without first adding this dependency.

File	pom.xml, dependencies section
Problem	spring-boot-starter-security not included in the project
OWASP Category	A02:2025 - Security Misconfiguration

30th Identified Problem - No dependency vulnerability scanning tool

There is also no OWASP Dependency Check plugin or similar tool in the build. This means the project does not automatically check for known vulnerabilities in its dependencies, leaving the project blind to newly disclosed CVEs.

File	pom.xml, build plugins
Problem	No dependency vulnerability analysis tool configured

OWASP Category	A03:2025 - Software Supply Chain Failures
-----------------------	---

4 Dynamic Analysis with OWASP ZAP

To test the security of the TutorialWS API, two tools were used. First, OWASP ZAP ran an automated scan on the app, checking all endpoints and detecting issues like missing security headers, insecure Basic Authentication and error messages that leak information. Then Postman was used to manually send crafted requests grouped by vulnerability type, testing things like unauthenticated access, SQL injection, XSS payloads and access to the H2 console. This confirmed the issues found by static analysis and the ZAP scan.

4.1 Testing with Postman

The Postman collection is organized into folders for each type of attack, so each group of requests targets a specific vulnerability. The table below summarizes what each test group does and its goal.

Table 13 - Postman tests

Folder / Scenario	Main Requests	Goal / Vulnerability Tested
SETUP	POST /tutorials GET /nifs POST /tutorials/{id}/comments	Pre-condition creates a base tutorial and comment for the following tests
Missing Authentication (CWE-306)	GET /tutorials GET /tutorials/{id} POST /tutorials PUT /tutorials/{id} DELETE /tutorials DELETE /tutorials/{id}	Checks that all endpoints are accessible without any credentials, proving the absence of authentication
Missing Auth NIFS	GET /nifs GET /nifs/{nif}	Tests access to the external NIFS service without authentication (skipped if NIFS is offline)
Anti-CSRF Tokens (CWE-352)	POST /tutorials PUT /tutorials/{id} DELETE /tutorials/{id} OPTIONS /tutorials	Confirms that write requests do not require a CSRF token, allowing cross-site attacks
Security Headers (CWE-693, CWE-1021)	GET /tutorials GET /h2-ui GET /nifs	Checks response headers: missing CSP, X-Frame-Options and X-Content-Type-Options
Improper input handling in external service call (potential abuse of outbound requests) (CWE-918)	POST /tutorials/{id}/comments with nif=0, -1, 9223372036854775807, 999999999	Sends extreme/invalid NIF values to trigger unauthorized requests to the external service (SSRF)
Injection: SQLi, XSS, Mass Assignment (CWE-89, CWE-79, CWE-915)	GET /tutorials?title=" OR "1"="1 POST /tutorials with XSS payloads (<script>, , <svg>) POST com campo id=9999	Tests SQL injection in query parameters, XSS in content fields and mass assignment via extra JSON fields

Anti-Automation (CWE-799)	GET /tutorials (multiple repetitions) GET /nifs (multiple repetitions)	Sends burst requests to check the absence of rate limiting or anti-automation mechanisms
Information Disclosure (CWE-200, CWE-209)	GET /tutorials/abc_invalido POST /tutorials {} GET /.git/config GET /application.properties OPTIONS /nifs	Triggers 500 errors to observe exception messages; attempts to access exposed sensitive files

4.2 Vulnerabilities Found by OWASP ZAP

The dynamic analysis with OWASP ZAP (version 2.17.0) was run against both <http://localhost:8080> and <http://pbs.dei.isep.ipp.pt:8080>. The scan found 10 alerts with different risk levels, shown in the table below.

Table 14 - OWASP ZAP vulnerabilities

Vulnerability	Affected Endpoint(s)	CWE	Summary Fix	Risk
Authentication Credentials Captured	GET /api/tutorials; POST /api/tutorials	287	Use HTTPS and replace Basic Auth with a secure mechanism (ex. JWT/OAuth2)	HIGH
Content Security Policy (CSP) Header Not Set	/h2-ui/ (GET, POST); PUT /api/tutorials/1 (pbs)	693	Configure the server to include the Content-Security-Policy header in all responses	MEDIUM
Missing Anti-clickjacking Header	/h2-ui/ (GET, POST)	1021	Add X-Frame-Options: DENY header or CSP frame-ancestors "none"	MEDIUM
Application Error Disclosure	/api/tutorials/1 (DELETE, GET); /api/tutorials (GET, POST)	550	Implement custom error pages; log details server-side without exposing them to the client	LOW
In Page Banner Information Leak	PUT /api/tutorials/1 (pbs) - exposed Tomcat/9.0.10 version	497	Hide the server version (Tomcat "server" attribute; custom error pages)	LOW
Timestamp Disclosure - Unix	GET /api/tutorials	497	Check if the timestamp is sensitive; if so, remove or mask it from the response	LOW
X-Content-Type-Options Header Missing	/api/tutorials (GET, POST); /nifs-app/nifs (pbs)	693	Add X-Content-Type-Options: nosniff header to all responses	LOW
Authentication Request Identified	POST /h2-ui/login.do	-	Informational alert; no action needed (used for automatic detection by ZAP)	INFO
Content-Type Header Missing	/api/tutorials (DELETE, OPTIONS, POST, PUT)	345	Make sure all responses explicitly set the Content-Type header	INFO
Modern Web Application	/h2-ui/ (GET, POST)	-	Informational alert; use ZAP Ajax Spider for better	INFO

			coverage of modern web applications	
--	--	--	-------------------------------------	--

5 Detailed Vulnerability Table

This section shows all the main security issues found in three of the four analyses done in this report: static analysis with SpotBugs (Section 3.1), STRIDE threat modelling (Section 2) and the dynamic testing with OWASP ZAP (Section 4).

The manual code review helped support the STRIDE analysis but isn't listed separately here since many of the findings are the same, all issues are listed by priority:

5.1 Severity Legend

Table 15 - ASVS: Severity legend

Severity	Meaning	Count	Distribution
CRITICAL	Immediate risk. Trivial exploitation. Full impact on confidentiality, integrity or availability.	3	ST-01, ST-02, ST-03
HIGH	Significant impact. Immediate attention after critical issues.	5	ST-04, ST-05, ST-06, ST-07, ST-08 ZAP: Authentication Credentials Captured
MEDIUM	Moderate risk. All 9 SpotBugs + 6 STRIDE threats.	14	SB-01 - SB-09, ST-09, ST-10, ST-11, ST-12, ST-14 ZAP: CSP Header, Anti-Clickjacking
LOW	Low impact. Address when convenient.	1	ST-13 ZAP: Error Disclosure, Server Version, Timestamp, X-Content-Type-Options

5.2 Main Vulnerability and Mitigation Tables

The following tables bring together all findings, the rows are grouped by discovery source and color-coded by severity. The ASVS column links each finding to the corresponding OWASP ASVS v4.0 requirement.

5.2.1 Automated Static Analysis

This table shows the 9 issues found with the tool SpotBugs in 338 lines of code, along with the affected file, the nature of each issue and the recommended fix, all rated Medium.

Table 16- ASVS: Automatic Static Analysis

ID	Component / File	Vulnerability / Threat	Type	Recommended Mitigation	ASVS
SB-01	Tutorial.java (line 72) Tutorial.removeTag(long)	Possible Null Pointer Dereference (NP_NULL_ON_SOME_PATH) The variable "tag" is checked for null at line 71 but dereferenced at line 72, which may throw a NullPointerException at runtime.	Correctness	Replace direct access with Optional.ofNullable(tag). Check for null before use: if (tag != null) { ... } Alternatively, refactor to use a stream with filter instead of manual iteration.	V5.1.3
SB-02	ErrorMessage.java (line 23) ErrorMessage.getTimestamp()	Expose Internal Representation: Return (EI_EXPOSE_REP) The getter returns a direct reference to the mutable internal Date field. An external caller can modify the timestamp and corrupt the object's internal state.	Malicious Code	Return a defensive copy: return new Date(this.timestamp.getTime()); Alternatively, migrate to java.time.Instant (immutable) instead of java.util.Date.	V8.3
SB-03	Comment.java (line 40) Comment.getTutorial()	Expose Internal Representation: Return (EI_EXPOSE_REP) Getter returns direct reference to the mutable Tutorial object associated with the Comment.	Malicious Code	Return a defensive copy or use a DTO: return new Tutorial(this.tutorial); Consider @JsonIgnore to prevent unintended exposure in API responses.	V8.3
SB-04	Tag.java (line 39) Tag.getTutorials()	Expose Internal Representation: Return (EI_EXPOSE_REP) Getter returns direct reference to the internal Set<Tutorial>, allowing callers to mutate the collection without control.	Malicious Code	Return an unmodifiable view: return Collections.unmodifiableSet(this.tutorials); Or a defensive copy: return new HashSet<>(this.tutorials);	V8.3
SB-05	Tutorial.java (line 76) Tutorial.getTags()	Expose Internal Representation: Return (EI_EXPOSE_REP) Getter returns direct reference to the internal Set<Tag>.	Malicious Code	Return an unmodifiable view: return Collections.unmodifiableSet(this.tags); Or a defensive copy: return new HashSet<>(this.tags);	V8.3
SB-06	ErrorMessage.java (line 13) new ErrorMessage(int, Date, String, String)	Expose Internal Representation: Store (EI_EXPOSE_REP2) Constructor stores the external Date reference directly. The caller can mutate the Date after construction and corrupt internal state.	Malicious Code	Store a defensive copy in the constructor: this.timestamp = new Date(timestamp.getTime()); Or migrate to java.time.Instant which is immutable by design.	V8.3
SB-07	Comment.java (line 44) Comment.setTutorial(Tutorial)	Expose Internal Representation: Store (EI_EXPOSE_REP2) Setter stores the external Tutorial reference directly. Mutations to the original object after setTutorial() will corrupt the Comment's internal state.	Malicious Code	Store a defensive copy: this.tutorial = new Tutorial(tutorial); If JPA context prevents copying, document the expected immutability contract.	V8.3
SB-08	Tag.java (line 42) Tag.setTutorials(Set)	Expose Internal Representation: Store (EI_EXPOSE_REP2) Setter stores the external Set<Tutorial> reference directly, exposing internal state to external mutation.	Malicious Code	Store a defensive copy: this.tutorials = new HashSet<>(tutorials); Ensure the internal collection is never replaced by an external reference.	V8.3
SB-09	Tutorial.java (line 80) Tutorial.setTags(Set)	Expose Internal Representation: Store (EI_EXPOSE_REP2) Setter stores the external Set<Tag> reference directly.	Malicious Code	Store a defensive copy: this.tags = new HashSet<>(tags); Consistent with the fix for SB-05 (defensive getter).	V8.3

5.2.2 Threat Analysis: STRIDE - Critical Severity

This table shows the 3 threats classified as Critical severity, through STRIDE threat modelling, representing the highest-priority vulnerabilities that must be addressed before anything else.

Table 17 - ASVS: Threat Analysis - Critical Severity

ID	Component / File	Vulnerability / Threat	Type	Recommended Mitigation	ASVS
ST-01	All REST endpoints (/api/tutorials /api/comments /api/tags)	Complete Absence of Authentication (Spoofing) Any anonymous client can create, edit and delete any resource. No role distinction or identity enforcement exists.	Spoofing	Add spring-boot-starter-security to pom.xml. Configure SecurityFilterChain with JWT or OAuth2. Annotate sensitive endpoints with @PreAuthorize("hasRole("USER")") and destructive operations with @PreAuthorize("hasRole("ADMIN")").	V4.1.1
ST-02	DELETE /api/tutorials (TutorialController .deleteAllTutorials)	Unprotected Mass Deletion (Tampering / DoS) A single unauthenticated DELETE request wipes the entire database. No confirmation, no role check.	Tampering / DoS	Protect with @PreAuthorize("hasRole("ADMIN")"). Implement soft-delete (deleted_at field). Add a mandatory confirmation token in the request body. Log all delete operations in an audit trail.	V4.1.3
ST-03	GET /h2-ui (application .properties)	H2 Console Publicly Exposed (Information Disclosure / EoP) spring.h2.console.enabled=true exposes a DB admin interface with no authentication. Allows arbitrary SQL execution over all application data.	Info Disclosure / EoP	In production: spring.h2.console.enabled=false. In development: spring.h2.console.settings.web-allow-others=false (restricts to localhost). If access is required, protect with Spring Security.	V14.2.2

5.2.3 Threat Analysis: STRIDE - High Severity

These 5 threats classified as High severity, should be addressed as soon as the Critical issues listed in the table above have been fixed.

Table 18 - ASVS: Threat Analysis: High Severity

ID	Component / File	Vulnerability / Threat	Type	Recommended Mitigation	ASVS
ST-04	application.properties (spring.datasource .password)	Empty / Plaintext Database Password (Information Disclosure) spring.datasource.password= is blank. Any process with filesystem access can connect to the H2 database without credentials.	Info Disclosure	Set a strong password: spring.datasource.password=\${DB_PASSWORD}. Use environment variables or Spring Vault for secret injection. Never commit credentials to VCS (add to .gitignore).	V2.10.1
ST-05	All REST endpoints (HTTP without TLS)	No HTTPS / TLS (Information Disclosure) All client-server communication runs over plaintext HTTP. NIFs, content and metadata are transmitted without encryption, vulnerable to interception (MITM).	Info Disclosure	Configure SSL/TLS in Spring Boot (server.ssl.*) with a valid certificate. Redirect all HTTP to HTTPS. Alternatively, place a reverse proxy (nginx/Traefik) with TLS termination in front of the application.	V9.1.1
ST-06	CommentController (PUT/DELETE /api/comments/{id})	No Ownership Verification (Elevation of Privilege) Any anonymous user can edit or delete another user's comments. No check whether the caller	EoP	After implementing authentication (ST-01), enforce ownership: @PreAuthorize("#comment.authorId == authentication.principal.id"). Return 403 Forbidden if the caller is not the resource owner.	V4.2.1

ID	Component / File	Vulnerability / Threat	Type	Recommended Mitigation	ASVS
		owns the comment being modified.			
ST-07	All REST endpoints (no rate limiting)	No Rate Limiting (Denial of Service) No request throttling per IP or user. An attacker can send thousands of requests per second, exhausting threads, memory and JDBC connections.	DoS	Integrate Bucket4j as a servlet filter. Limit per IP: max 100 req/min for GET, 20/min for POST/PUT/DELETE. Alternative: Spring Cloud Gateway with RequestRateLimiter filter.	V11.1.2
ST-08	CommentController (Optional.of(nif) nif potentially null)	NullPointerException via Optional.of(null) (Denial of Service) When the NIF Service is unavailable, onErrorReturn(null) returns null and the subsequent Optional.of(nif) throws an unhandled NullPointerException.	DoS	Replace Optional.of(nif) with Optional.ofNullable(nif). Return 503 Service Unavailable when NIF Service is down. Implement a Resilience4j circuit breaker with fallback.	V11.1.2

5.2.4 Threat Analysis: STRIDE - Medium Severity

These 5 Medium severity issues have moderate impact and should be planned for the current development cycle.

Table 19 - ASVS: Threat Analysis - Medium Severity

ID	Component / File	Vulnerability / Threat	Type	Recommended Mitigation	ASVS
ST-09	TutorialController CommentController (@RequestBody no @Valid)	Missing Input Validation (Tampering) Fields title, description and content accept null, empty or arbitrarily large values. @Lob on content allows oversized data injection into the database.	Tampering	Add spring-boot-starter-validation. Annotate parameters with @Valid. Add @NotBlank, @Size(max=500) on title/description and @Size(max=2000) on content. Handle MethodArgumentNotValidException globally.	V5.1.3
ST-10	WebFluxConfig.java (hardcoded NIF Service URL)	Hardcoded External Service URL (Information Disclosure) http://pbs.dei.isep.ipp.pt:8080/nifs-app is hardcoded in source. Exposes internal infrastructure in the repository. Communication over plaintext HTTP.	Info Disclosure	Move URL to application.properties: nif.service.base-url=https://... Inject with @Value("\${nif.service.base-url}"). Change http:// to https://. Configure response timeout of 5 seconds on the HttpClient.	V2.10.1/ V9.1.1
ST-11	WebFluxConfig.java (WebClient without timeout)	No Timeout on WebClient (Denial of Service) No responseTimeout, connectionTimeout or readTimeout is configured. If the NIF Service becomes unresponsive, requests block indefinitely and exhaust the thread pool.	DoS	Configure HttpClient with timeout: HttpClient.newConnection().responseTimeout(Duration.ofSeconds(5)).option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3000). Implement Resilience4j circuit breaker with fallback.	V11.1.2
ST-12	application.properties (ddl-auto=update)	DDL Auto-Update Enabled in Production (Tampering) Hibernate can automatically alter the DB schema on every startup. In production, this may cause data loss or schema inconsistency between versions.	Tampering	Change to ddl-auto=validate (checks schema without altering) or none in production. Manage schema migrations with Flyway or Liquibase for full version control.	V14.2.2

ID	Component / File	Vulnerability / Threat	Type	Recommended Mitigation	ASVS
ST-14	All controllers (no logging or auditing)	No Logging or Audit Trail (Repudiation) No write operations (POST, PUT, DELETE) are logged. Impossible to audit who created, modified or deleted resources. No incident traceability.	Repudiation	Implement SLF4J/Logback. Log all write operations with: caller IP, operation, affected resource and timestamp. After authentication: include the authenticated user ID. Consider Spring Data Envers for entity-level audit.	V7.3.1

5.2.5 Threat Analysis: STRIDE - Low Severity

Finally, a minor configuration issue, classified as Low severity, should be fixed to keep the codebase consistent and maintainable.

Table 20 - ASVS: Threat Analysis - Low Severity

ID	Component / File	Vulnerability / Threat	Type	Recommended Mitigation	ASVS
ST-13	TutorialController TagController (@CrossOrigin)	Inconsistent CORS Policy (Spoofing) TutorialController and CommentController use @CrossOrigin(localhost:8080), while TagController uses localhost:8081. Fragmented, error-prone configuration.	Spoofing	Remove all per-controller @CrossOrigin annotations. Centralise CORS in WebMvcConfigurer or SecurityFilterChain. Define explicit allowed origins per environment profile (dev/prod).	V14.5.3

5.2.6 Dynamic Analysis: OWASP ZAP Findings

This table shows the security issues found by OWASP ZAP during dynamic analysis. In this case, informational alerts are excluded since they require no action and where a finding overlaps with a STRIDE entry, the corresponding ID is indicated.

Table 21 - ASVS: Dynamic Analysis - OWASP ZAP

Vulnerability	Affected Endpoints	CWE	Recommended Mitigation	Related
Authentication Credentials Captured	GET /api/tutorials POST /api/tutorials	CWE-287	Enforce HTTPS (ST-05) and replace any Basic Auth with JWT or OAuth2 (ST-01).	ST-01, ST-05
Content Security Policy (CSP) Header Not Set	/h2-ui/ PUT /api/tutorials/1	CWE-693	Add Content-Security-Policy header via Spring Security: .headers().contentSecurityPolicy("default-src 'self'").	V14.4.3
Missing Anti-Clickjacking Header	/h2-ui/ (GET, POST)	CWE-1021	Add X-Frame-Options: DENY via Spring Security .headers() to prevent clickjacking attacks.	V14.4.6
Application Error Disclosure	/api/tutorials /api/tutorials/1	CWE-550	Replace ex.getMessage() in ControllerExceptionHandler with a generic message. Log full detail server-side only.	Manual #25, #26
Server Version Disclosure (Tomcat)	PUT /api/tutorials/1 (pbs)	CWE-497	Hide server version by configuring the Tomcat server attribute and using custom error pages.	

Vulnerability	Affected Endpoints	CWE	Recommended Mitigation	Related
Timestamp Disclosure	GET /api/tutorials	CWE-497	Review whether timestamps in responses are sensitive. If so, remove or mask them from the response body.	
X-Content-Type-Options Header Missing	/api/tutorials /nifs-app/nifs (pbs)	CWE-693	Add X-Content-Type-Options: nosniff via Spring Security .headers(). Enabled by default when .headers() is configured.	V14.4.4

5.3 ASVS Level 2 Gap Analysis

The OWASP Application Security Verification Standard v4.0 defines three levels of security verification. The Level 2 is the recommended baseline for applications that handle sensitive data and the table below checks the application against the most relevant requirements to a REST API of this type.

The application meets only 1 requirement, with 18 missing and 3 partially implemented, the main gaps are authentication, encrypted communications and secure configuration and fixing these would be the first step towards making the application production-ready.

Table 22 - ASVS Level 2 Gap Analysis

ASVS	Chapter	Requirement	Status	How to Address
V2.10.1	Authentication	No hardcoded credentials in source or config files	Missing	Move DB password and NIF service URL out of source code into environment variables or a secrets manager such as Spring Vault. Never commit credentials to VCS.
V4.1.1	Access Control	All components enforce access control	Missing	Add spring-boot-starter-security and configure SecurityFilterChain with JWT or OAuth2 (deny by default).
V4.1.3	Access Control	Least privilege applied	Missing	Define USER and ADMIN roles via Spring Security. Restrict destructive operations such as DELETE all to ADMIN using @PreAuthorize("hasRole('ADMIN')").
V4.1.5	Access Control	Deny by default	Missing	Configure Spring Security with a deny-by-default policy so all requests are blocked unless explicitly permitted by a security rule.
V4.2.1	Access Control	Object-level authorization (IDOR)	Missing	After implementing authentication, verify comment ownership before allowing PUT/DELETE on /api/comments/{id}. Return 403 Forbidden if the caller is not the resource owner.
V4.2.2	Access Control	CSRF protection	Missing	Enable Spring Security CSRF tokens for session-based auth. If using stateless JWT, CSRF is not required as Bearer tokens are not automatically sent by browsers.

ASVS	Chapter	Requirement	Status	How to Address
V5.1.3	Validation	Input validated server-side	Missing	Add spring-boot-starter-validation. Annotate @RequestBody parameters with @Valid and add @NotBlank and @Size constraints on entity fields such as title, description and content.
V5.3.1	Encoding	XSS protection	Missing	Confirmed by Postman tests, XSS payloads were stored without sanitisation. Apply OWASP Java HTML Sanitizer on content fields before persisting to the database.
V7.1.1	Logging	No sensitive data in logs	Partial	Replace System.out.println in CommentController with SLF4J/Logback. Configure log patterns to mask or exclude sensitive fields such as NIF values.
V7.3.1	Logging	Logging done server-side	Missing	Implement SLF4J/Logback. Log all write operations with caller IP, operation, affected resource and timestamp. After authentication is added, include the authenticated user ID.
V7.4.1	Error Handling	No internal details exposed	Missing	ControllerExceptionHandler currently returns ex.getMessage() directly in the response body. Replace with a generic message and log the full exception detail server-side only.
V9.1.1	Communications	HTTPS enforced	Missing	Configure server.ssl.* in application.properties with a valid certificate. Redirect all HTTP traffic to HTTPS. Alternatively, place a reverse proxy such as nginx with TLS termination in front of the application.
V9.1.2	Communications	Strong TLS only	Missing	Set server.ssl.enabled-protocols=TLSv1.2,TLSv1.3 in application.properties when implementing V9.1.1. Disable TLS 1.0 and 1.1 explicitly.
V11.1.2	Business Logic	Rate limiting	Missing	Integrate Bucket4j as a servlet filter. Set limits per IP: max 100 requests/min for GET and 20 requests/min for POST, PUT and DELETE operations.
V13.1.2	API Security	Access control on all endpoints	Missing	Dependent on V4.1.1 :the Spring Security filter chain must cover all /api/* paths to ensure no endpoint is left unprotected.
V13.2.2	API Security	JSON validation	Missing	Use @Valid with Bean Validation constraints on all @RequestBody parameters. Consider JSON Schema validation for more complex or nested payloads.
V14.2.1	Configuration	Up-to-date components	Missing	The project uses Spring Boot 2.6.7 released in 2022, which has known CVEs. Upgrade to Spring Boot 3.x and review dependency versions in pom.xml.

ASVS	Chapter	Requirement	Status	How to Address
V14.2.2	Configuration	Unnecessary features disabled	Missing	Set <code>spring.h2.console.enabled=false</code> in the production profile. Remove or restrict Spring Actuator endpoints to prevent exposure of internal diagnostic information.
V14.4.1	HTTP Headers	Content-Type present	Partial	ZAP confirmed Content-Type is missing on DELETE and OPTIONS responses. Configure Spring to include the Content-Type header explicitly on all responses.
V14.4.3	HTTP Headers	CSP header present	Missing	Add a Content-Security-Policy header via Spring Security: <code>.headers().contentSecurityPolicy("default-src 'self'")</code> . This mitigates XSS and data injection attacks confirmed by dynamic analysis.
V14.4.4	HTTP Headers	X-Content-Type-Options	Missing	Enable X-Content-Type-Options: nosniff. This is included by default when Spring Security <code>.headers()</code> is configured, preventing MIME type sniffing attacks.
V14.4.6	HTTP Headers	X-Frame-Options	Partial	ZAP confirmed X-Frame-Options is missing on H2 console endpoints. Enable DENY or SAMEORIGIN via Spring Security <code>.headers()</code> to prevent clickjacking.
V14.5.3	HTTP Headers	CORS allowlist validated server-side	OK	Remove per-controller <code>@CrossOrigin</code> annotations. Centralise CORS in <code>WebMvcConfigurer</code> or <code>SecurityFilterChain</code> with an explicit server-side allowlist of trusted origins.

6 Mitigation Strategies and Security Implementation

6.1 Missing Content Security Policy (CSP) header

Severity: MEDIUM (Mitigates XSS but not primary control), CVSS Score: 5.3, Effort to Fix: Low (1-2 hours)

This project is a no frontend JS app, so Spring never sets Content-Security-Policy headers by default. ZAP is still scanning `/api/tutorials` and is expecting CSP on any HTTP response served by the web server. If the API response is ever consumed by the browser, it becomes easier to exploit an XSS vulnerability. CSP is a strong defence-in-depth control and it's missing by default. This can be fixed adding a security header in Spring Boot. Example:

```
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .headers()
                .contentSecurityPolicy("default-src 'self'");
    }
}
```

Figure 6 - Recommended Implementation to CSP

6.2 Missing Anti-clickjacking header

Severity: MEDIUM (Admin-targeted attack), CVSS Score: 4.3, Effort to Fix: Low (30 minutes)

ZAP is reporting that Anti-clickjacking header is missing for /h2-ui/ and .../login.do. Spring Boot/Tomcat response header do not include X-Frame-Options: DENY/SAMEORIGIN or even Content-Security-Policy: frame-ancestors allowing the application to be framed by another website, making it possible to clickjack. Being more precise, a malicious site could embed this app in an <iframe> and trick admin users into clicking buttons or submitting forms. This can be mitigated by setting headers for all pages such as `response.setHeader("X-Frame-Options", "DENY");` or even `response.setHeader("Content-Security-Policy", "frame-ancestors 'none'");`.

6.3 Application Error disclosure

Severity: CRITICAL (Information disclosure), CVSS Score: 7.5, Effort to Fix: Medium (2-4 hours, includes refactoring)

ZAP reports that Application Error Disclosure via /api/tutorials POST returns HTTP 500 with details. The app has a global handler (ControllerExceptionHandler in ControllerExceptionHandler.java) that produces a JSON object but if we look at the code, if an exception is thrown that isn't being handled, Spring Boot will still produce an error page/error JSON that might include stack traces/internal details.

This is happening since CommentController.createComment() (CommentController.java) calls external API findById ((Long)commentRequest.getNif()) and can throw an exception or a blocked response. Tutorial.RemoveTag (in Tutorial.java line 69) has a NPE risk for missing tag that could surface as a 500.

This exposed exception messages/stack traces can reveal framework versions, package names or even internal logic making attackers being able to target known vulnerable code. This can be mitigated if safe error response is used (example in TutorialWS\src\main\java\com\example\tutorialws\exception\ErrorMessage.java) but ensuring no raw exception is leaked returning only a generic message to clients and logging the exception internally with details for debugging in the server log.

Mitigation can also be done with never exposing stack trace/exception messages in response: in TutorialWS\src\main\resources\application.properties the following lines should be added:

- server.error.include-stacktrace=never
- server.error.include-message=never

- `server.error.include-binding-errors=never`

6.4 In-page banner/Tomcat Version Leak

Severity: LOW (Reconnaissance only), CVSS Score: 3.7, Effort to Fix: Low (30 minutes)

ZAP reports that Tomcat/9.0.10 is detected in responses since Tomcat (Spring Boot embedded) is sending the `Server: Apache-Coyote/1.1` header and a response banner that includes Tomcat/9.0.10 that is the default behaviour of Tomcat/Spring Boot. With this information, attackers can target known vulnerabilities specific to Tomcat 9.0.10 or the Spring Boot version. This can be avoided if server version header is suppressed adding in `TutorialWS\src\main\resources\application.properties` `server.server-header=# empty` value disables header and disabling banner output `spring.main.banner-mode=off`. Patching to a modern Spring Boot/Tomcat version that has known security issues fixed can be a way of mitigating this problem.

6.5 Timestamp disclosure

Severity: LOW (Fingerprinting aid), CVSS Score: 3.1, Effort to Fix: Low (1 hour)

ZAP saw a Unix timestamp (1773876317) in a response and flagged it. This is possible since the app is returning timestamps either by via error response payloads or directly: the `ErrorMessage` DTO in

`TutorialWS\src\main\java\com\example\tutorialws\exception\ErrorMessage.java` includes a timestamp field and `ControllerExceptionHandler` in `TutorialWS\src\main\java\com\example\tutorialws\exception\ControllerExceptionHandler.java` always sets `new Date()`. Timestamps can leak deployment timing or application activity patterns that can be useful in combined attacks or fingerprinting. The mitigation for this issue involves removing the timestamp from the response if it is not needed by consumers or if it is needed, not include it in a public API response and only keep it in logs.

```
public class ErrorMessage {
    private int statusCode;
    private Date timestamp;
    private String message;
    private String description;

    public ErrorMessage(int statusCode, Date timestamp, String message, String description) {
        this.statusCode = statusCode;
        this.timestamp = timestamp;
        this.message = message;
        this.description = description;
    }
}
```

Figure 7 - Current code of ErrorMessage.java

```
public class ErrorMessage {
    private int statusCode;
    private String message;
    private String description;

    public ErrorMessage(int statusCode, String message, String description) {
        this.statusCode = statusCode;
        this.message = message;
        this.description = description;
    }
}
```

Figure 8 - Recommended implementation of ErrorMessage.java

```
@ExceptionHandler(value = ResourceNotFoundException.class)
public ResponseEntity<ErrorMessage> resourceNotFoundException(
    ResourceNotFoundException ex, HttpServletRequest request) {
    ErrorMessage em = new ErrorMessage(
        HttpStatus.NOT_FOUND.value(),
        ex.getMessage(),
        request.getServletPath());
    // timestamp not exposed, logged internally instead
    log.error("Resource not found: {} | Timestamp: {}",
        ex.getMessage(), System.currentTimeMillis());
    return new ResponseEntity<>(em, HttpStatus.NOT_FOUND);
}
```

Figure 9 -Updated ControllerExceptionHandler.java

6.6 X-Content-Type-Options missing

Severity: MEDIUM (MIME sniffing), CVSS Score: 5.1, Effort to Fix: Low (1-2 hours)

ZAP reports that X-Content-Type-Options header is missing and this happens because Spring Boot doesn't add this header by default for all responses. This is a risk since Browsers can "MIME Sniff" content, treating a JSON response as HTML/executable, enabling XSS or drive-by downloads. If headers such as `http.headers().xssProtection().and().contentTypeOptions()` (via Spring Security) or `response.setHeader("X-Content-Type-Options", "nosniff")` (via filter) are added globally, this can be avoided.

6.7 Missing Content-Type header

Severity: LOW (Spec compliance), CVSS Score: 2.7, Effort to Fix: Medium (1-2 hours)

ZAP notes that Content-Type is missing on some DELETE/OPTIONS requests because Spring Boot typically sets Content-Type: application/Json for JSON bodies but DELETE responses or responses with empty bodies cannot set it and if `ResponseEntity` is return without body, Sprint will omit it. This is solved by ensuring controllers always set an explicit Content-Type.

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteComment(@PathVariable Long id) {
    commentService.delete(id);
    return ResponseEntity
        .status(HttpStatus.NO_CONTENT)
        .header("Content-Type", "application/json")
        .build();
}
```

Figure 10 - Recommended pattern to ResponseEntity

```
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteComment(@PathVariable Long id) {
    commentService.delete(id);
}
```

Figure 11 - Alternative using @ResponseStatus

Global Configuration (application.properties):

- spring.mvc.contentnegotiation.parameter-name=format
- server.servlet.default-servlet-name=default

6.8 H2 Console exposed

Severity: CRITICAL (Complete DB access), CVSS Score: 9.1, Effort to Fix: Low (30 minutes)

The H2 console (/h2-ui/) is accessible and was scanned by ZAP and it's explicitly enabled (*spring.h2.console.enabled=true* in TutorialWS\src\main\resources\application.properties) allowing that anyone can drop tables, read data, ... and it's often accessible with default credentials up to trivial configurations. This is mitigated by removing *spring.h2.console.enabled=true* or setting it conditionally if dev profile is being used with authentication protection and restricting IP access.

6.9 Additional failures

6.9.1 GeneratedValue

Severity: CRITICAL (Application startup failure), CVSS Score: 6.5, Effort to Fix: Low (1 hour)

This JPA should have a sequence generator (common failure) since without it, at startup it can fail leading to hibernation. With this, it is recommended to define a sequence generator or use IDENTITY for H2.

The entities Comment.java, Tag.java and Tutorial.java uses

```
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "comment_generator")
```

Figure 12 - Current code used by different entities

But @SequenceGenerator annotation is never defined leading to a Hibernate/JPA configuration error. This JPA with strategy=SEQUENCE tells Hibernate to use a database sequence for auto-generating primary key values but it's needed to define the sequence via @SequenceGenerator(name = "...", sequenceName = "...", ...) since without this, Hibernate cannot resolve the generator name at runtime. This happens during the JPA bootstrap phase before any HTTP requests are handled.

```
@Entity
@Table(name = "tutorials")
public class Tutorial {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private long id;
    // ... rest of code
```

Figure 13 - Current code of Tutorial class

```
@Entity
@Table(name = "tutorials")
public class Tutorial {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "tutorial_seq")
    @SequenceGenerator(
        name = "tutorial_seq",
        sequenceName = "tutorial_sequence",
        allocationSize = 1)
    private long id;
    // ... rest of code
}
```

Figure 14 - Improved code of Tutorial class

```
CREATE SEQUENCE tutorial_sequence
START WITH 1
INCREMENT BY 1;

CREATE TABLE tutorials (
    id BIGINT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description VARCHAR(2000),
    published BOOLEAN DEFAULT FALSE
);
```

Figure 15 - Database Schema

6.9.2 Tutorial.removeTag has a NPE

Severity: MEDIUM (Denial of Service), CVSS Score: 5.3, Effort to Fix: Low (1 hour)

If no tag with the give tagId exists in this.tags, tag is null but the method still calls the function remove(this), causing a NPE.

```
Tag tag = this.tags.stream().filter(t -> t.getId() == tagId).findFirst().orElse(null);
if (tag != null) this.tags.remove(tag);
tag.getTutorials().remove(this); // NPE if tag == null
```

Figure 16 - Current Code of Tutorial.removeTag

The if (tag != null) only protects remove(tag) since the following line is outside the if block. The improved code would be:

```
if (tag != null) {
    this.tags.remove(tag);
    tag.getTutorials().remove(this);
}
```

Figure 17 - Recommended code to Tutorial.removeTag

7 Conclusion

With this report, we looked at the security of the TutorialWS REST API and found multiple serious weaknesses across the system. The results show that the application is missing some basic security controls, especially in authentication, access control, input validation and data protection. All these issues make it easy for an attacker to access, modify or delete data without effort.

As of right now, the application is not ready to be used in real production environment, the issues found show that secure design practices were not considered during development and to improve security, the first step should be fixing the most critical problems, such as adding proper authentication and restricting access to sensitive operations. After that, other improvements like input validation, secure communication and better logging should also be added.

Overall, this analysis gives a clear idea of where the main problems are and what needs to be done next, fixing these issues will significantly improve the overall security and make the system more reliable.

8 References

- [1] OWASP Foundation, "Threat Modeling Process," OWASP Community, 2021. [Online]. Available: https://owasp.org/www-community/Threat_Modeling_Process. [Accessed: Mar. 2026].
- [2] OWASP Foundation, "OWASP Top 10:2025," 2025. [Online]. Available: <https://owasp.org/Top10/2025/>. [Accessed: Mar. 2026].
- [3] OWASP Foundation, "Application Security Verification Standard (ASVS) v4.0.3," 2021. [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>. [Accessed: Mar. 2026].
- [4] OWASP Foundation, "OWASP ZAP (Zed Attack Proxy) v2.17.0," 2024. [Online]. Available: <https://www.zaproxy.org/>. [Accessed: Mar. 2026].
- [5] SpotBugs Development Team, "SpotBugs Static Analysis Tool for Java," Version 4.8.6, 2024. [Online]. Available: <https://spotbugs.github.io/>. [Accessed: Mar. 2026].
- [6] Spring Boot Team, "Spring Boot Reference Documentation," Version 2.6.7, VMware, Inc., 2022. [Online]. Available: <https://docs.spring.io/spring-boot/docs/2.6.7/reference/html/>. [Accessed: Mar. 2026].
- [7] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014. ISBN: 978-1118809990.
- [8] M. Howard and S. Lipner, The Security Development Lifecycle. Redmond, WA, USA: Microsoft Press, 2006. ISBN: 978-0735622142.
- [9] OWASP Foundation, "OWASP API Security Top 10:2023," 2023. [Online]. Available: <https://owasp.org/www-project-api-security/>. [Accessed: Mar. 2026].
- [10] MITRE Corporation, "Common Weakness Enumeration (CWE) List," Version 4.14, 2024. [Online]. Available: <https://cwe.mitre.org/>. [Accessed: Mar. 2026].
- [11] Resilience4j Authors, "Resilience4j Fault Tolerance Library for Java," 2024. [Online]. Available: <https://resilience4j.readme.io/>. [Accessed: Mar. 2026].
- [12] P. B. Sousa, "M2: Project Assignment Brief," ISEP PGSICP ADDSS, 2025/2026. [Internal document].
- [13] OWASP Foundation, "OWASP Threat Dragon," 2024. [Online]. Available: <https://owasp.org/www-project-threat-dragon/>. [Accessed: Mar. 2026].
- [14] Bucket4j Authors, "Bucket4j Rate Limiting Library for Java," 2024. [Online]. Available: <https://bucket4j.com/>. [Accessed: Mar. 2026].
- [15] OWASP Foundation, "OWASP Java HTML Sanitizer," 2024. [Online]. Available: <https://owasp.org/www-project-java-html-sanitizer/>. [Accessed: Mar. 2026].

9 Annex 1 - TutorialWS_DFD

TutorialWS - DFD Multi-diagrama com Ameaças STRIDE

Owner: Jose Rodrigues, Carlos Silva, Sara Bastos, Marta Portugal
Reviewer: Prof. Paulo Baltarejo
Contributors:
Date Generated: Tue Mar 17 2026

High level system description

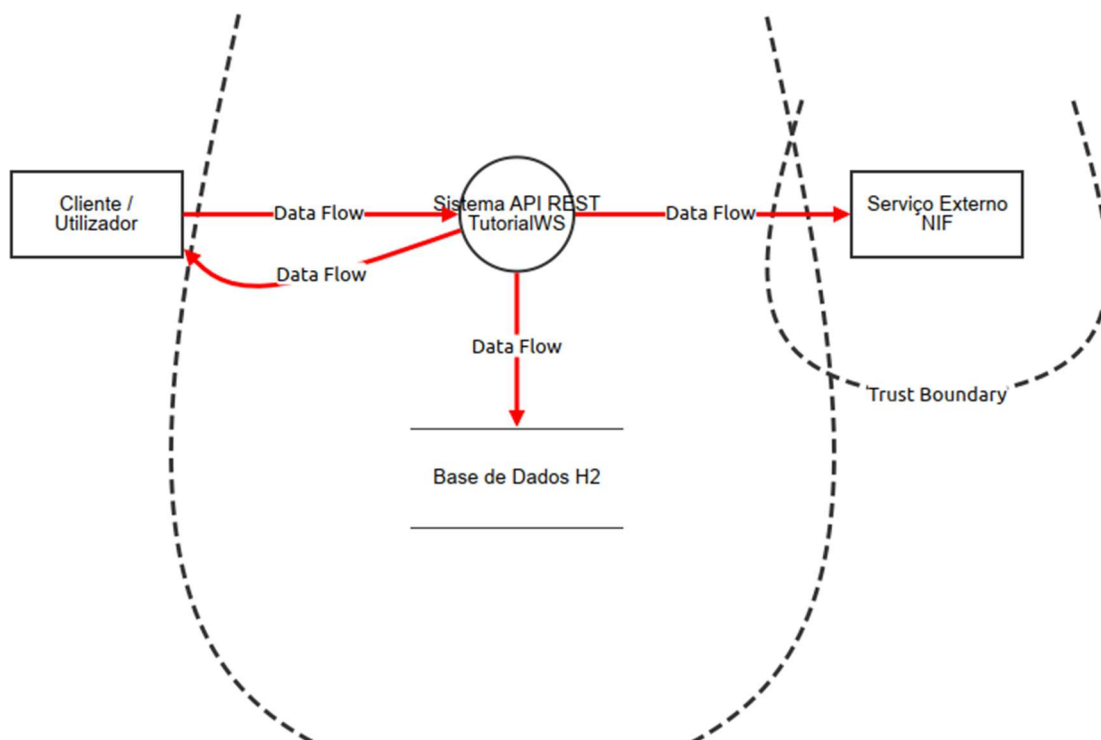
Modelo Threat Dragon com DFD de contexto, nível 1 e processos detalhados para TutorialWS. Inclui trust boundaries e ameaças

Summary

Total Threats	44
Total Mitigated	0
Total Open	44
Open / Critical Severity	0
Open / High Severity	14
Open / Medium Severity	27
Open / Low Severity	3

DFD Contexto

Visão global do sistema TutorialWS com principais entidades, stores, trust boundaries e ameaças.



DFD Contexto

Cliente / Utilizador (Actor) - *Out of Scope*

Reason for out of scope: Entidade externa ao sistema

Description: Entidade externa que consome a API REST.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Sistema API REST TutorialWS (Process)

Description: Aplicação Spring Boot que expõe endpoints para tutoriais, comentários e tags.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Ausência de autenticação nos endpoints	Spoofing	High	Open		Os endpoints REST podem ser invocados sem autenticação, permitindo ações por entidades não autorizadas.	Introduzir autenticação forte e autorização por perfil ou recurso.
2	Tratamento genérico de erros pode expor comportamento interno	Information disclosure	Medium	Open		Mensagens de erro e códigos de resposta podem facilitar a enumeração de recursos e o conhecimento do comportamento da aplicação.	Uniformizar respostas de erro e reduzir detalhe devolvido ao cliente.

10 Annex 2 - SpotBugs Report

SpotBugs Report

file:///C:/Users/cms/OneDrive%20-%20sisint.pt/_Pos%20Gradua%C...

SpotBugs Report

Produced using [SpotBugs](#)4.8.6.

Project: m2[TutorialWS]

Metrics

338 lines of code analyzed, in 15 classes, in 6 packages.

Metric	Total	Density*
High Priority Warnings		NaN
Medium Priority Warnings	9	26.63
Total Warnings	9	26.63

(* Defects per Thousand lines of non-commenting source statements)

Summary

Warning Type	Number
Correctness Warnings	1
Malicious code vulnerability Warnings	8
Total	9

Warnings

Click on each warning link to see a full description of the issue, and details of how to resolve it.

Correctness Warnings

Warning	Priority	Details
Possible null pointer dereference	Medium	Possible null pointer dereference of tag in

com.example.tutorialws.model.Tutorial.removeTag(long)

In file Tutorial.java Tutorial.java, line 72 71
In class com.example.tutorialws.model.Tutorial
In method com.example.tutorialws.model.Tutorial.removeTag(long)
Value loaded from tag
Dereferenced at Tutorial.java:[line 72]
Known null at Tutorial.java:[line 71]

Malicious code vulnerability Warnings

Warning	Priority	Details
May expose internal representation by returning reference to mutable object	Medium	com.example.tutorialws.exception.ErrorMessage.getTimestamp() may expose internal representation by returning ErrorMessage.timestamp In file ErrorMessage.java ErrorMessage.java, line 23 23 In class com.example.tutorialws.exception.ErrorMessage In method com.example.tutorialws.exception.ErrorMessage.getTimestamp() Field com.example.tutorialws.exception.ErrorMessage.timestamp At ErrorMessage.java:[line 23] At ErrorMessage.java:[line 23]
May expose internal representation by returning reference to mutable object	Medium	com.example.tutorialws.model.Comment.getTutorial() may expose internal representation by returning Comment.tutorial In file Comment.java Comment.java, line 40 40 In class com.example.tutorialws.model.Comment In method com.example.tutorialws.model.Comment.getTutorial() Field com.example.tutorialws.model.Comment.tutorial At Comment.java:[line 40] At Comment.java:[line 40]
May expose internal representation by returning reference to mutable object	Medium	com.example.tutorialws.model.Tag.getTutorials() may expose internal representation by returning Tag.tutorials In file Tag.java Tag.java, line 39 39

		In class com.example.tutorialws.model.Tag In method com.example.tutorialws.model.Tag.getTutorials() Field com.example.tutorialws.model.Tag.tutorials At Tag.java:[line 39] At Tag.java:[line 39]
May expose internal representation by returning reference to mutable object	Medium	com.example.tutorialws.model.Tutorial.getTags() may expose internal representation by returning Tutorial.tags In file Tutorial.java Tutorial.java, line 76 76 In class com.example.tutorialws.model.Tutorial In method com.example.tutorialws.model.Tutorial.getTags() Field com.example.tutorialws.model.Tutorial.tags At Tutorial.java:[line 76] At Tutorial.java:[line 76]
May expose internal representation by incorporating reference to mutable object	Medium	new com.example.tutorialws.exception.ErrorMessage(int, Date, String, String) may expose internal representation by storing an externally mutable object into ErrorMessage.timestamp In file ErrorMessage.java ErrorMessage.java, line 13 13 In class com.example.tutorialws.exception.ErrorMessage In method new com.example.tutorialws.exception.ErrorMessage(int, Date, String, String) Field com.example.tutorialws.exception.ErrorMessage.timestamp Local variable named timestamp At ErrorMessage.java:[line 13] At ErrorMessage.java:[line 13]
May expose internal representation by incorporating reference to mutable object	Medium	com.example.tutorialws.model.Comment.setTutorial(Tutorial) may expose internal representation by storing an externally mutable object into Comment.tutorial In file Comment.java Comment.java, line 44 44 In class com.example.tutorialws.model.Comment In method com.example.tutorialws.model.Comment.setTutorial(Tutorial) Field com.example.tutorialws.model.Comment.tutorial Local variable named tutorial At Comment.java:[line 44] At Comment.java:[line 44]
May expose internal	Medium	com.example.tutorialws.model.Tag.setTutorials(Set) may expose internal representation by storing an externally mutable object into Tag.tutorials

[representation by
incorporating
reference to
mutable object](#)

In file Tag.java Tag.java, line 42 42
In class com.example.tutorialws.model.Tag
In method com.example.tutorialws.model.Tag.setTutorials(Set)
Field com.example.tutorialws.model.Tag.tutorials
Local variable named tutorials
At Tag.java:[line 42]
At Tag.java:[line 42]

[May expose
internal
representation by
incorporating
reference to
mutable object](#)

Medium

com.example.tutorialws.model.Tutorial.setTags(Set) may expose internal
representation by storing an externally mutable object into Tutorial.tags

In file Tutorial.java Tutorial.java, line 80 80
In class com.example.tutorialws.model.Tutorial
In method com.example.tutorialws.model.Tutorial.setTags(Set)
Field com.example.tutorialws.model.Tutorial.tags
Local variable named tags
At Tutorial.java:[line 80]
At Tutorial.java:[line 80]

Warning Types

May expose internal representation by returning reference to mutable object

Returning a reference to a mutable object value stored in one of the object's fields exposes the internal representation of the object. If instances are accessed by untrusted code, and unchecked changes to the mutable object would compromise security or other important properties, you will need to do something different. Returning a new copy of the object is better approach in many situations.

May expose internal representation by incorporating reference to mutable object

This code stores a reference to an externally mutable object into the internal representation of the object. If

instances are accessed by untrusted code, and unchecked changes to the mutable object would compromise security or other important properties, you will need to do something different. Storing a copy of the object is better approach in many situations.

Possible null pointer dereference

There is a branch of statement that, *if executed*, guarantees that a null value will be dereferenced, which would generate a `NullPointerException` when the code is executed. Of course, the problem might be that the branch or statement is infeasible and that the null pointer exception cannot ever be executed; deciding that is beyond the ability of SpotBugs.

11 Annex 3 - ZAP Report



Sites: <http://pbs.dei.isep.ipp.pt:8080> <http://localhost:8080>

Generated on quinta, 19 mar. 2026 19:11:21

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
Alto	1
Médio	2
Baixo	4
Informational	3

Insights

Level	Reason	Site	Description	Statistic
Baixo	Aviso		ZAP errors logged - see the zap. log file for details	8
Baixo	Aviso		ZAP warnings logged - see the zap. log file for details	9
Info	Informational		Percentage of network failures	8 %
Info	Informational	http://localhost:8080	Percentage of responses with status code 2xx	58 %
Info	Informational	http://localhost:8080	Percentage of responses with status code 3xx	2 %
Info	Informational	http://localhost:8080	Percentage of responses with status code 4xx	24 %
Info	Informational		Percentage of responses	16 %

		http://localhost:8080	with status code 5xx	
Info	Informational	http://localhost:8080	Percentage of endpoints with content type application/json	76 %
Info	Informational	http://localhost:8080	Percentage of endpoints with content type text/html	4 %
Info	Informational	http://localhost:8080	Percentage of endpoints with content type text/plain	2 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method DELETE	14 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method GET	54 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method OPTIONS	4 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method POST	20 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method PUT	8 %
Info	Informational	http://localhost:8080	Count of total endpoints	50
Info	Informational	http://localhost:8080	Percentage of slow responses	25 %
			Percentage of	

Info	Informational	http://pbs.dei.isep.ipp.pt:8080	responses with status code 2xx	17 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of responses with status code 4xx	82 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of endpoints with content type application/json	12 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of endpoints with content type text/html	72 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of endpoints with content type text/plain	9 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of endpoints with method DELETE	12 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of endpoints with method GET	69 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of endpoints with method POST	12 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of endpoints with method PUT	6 %
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Count of total endpoints	33
Info	Informational	http://pbs.dei.isep.ipp.pt:8080	Percentage of slow responses	3 %

Alertas

Name	Risk Level	Number of Instances
Authentication Credentials Captured	Alto	2
Content Security Policy (CSP) Header Not Set	Médio	Systemic
Missing Anti-clickjacking Header	Médio	2
Application Error Disclosure	Baixo	4
In Page Banner Information Leak	Baixo	Systemic
Timestamp Disclosure - Unix	Baixo	1
X-Content-Type-Options Header Missing	Baixo	Systemic
Authentication Request Identified	Informational	1
Content-Type Header Missing	Informational	4
Modern Web Application	Informational	3

Alert Detail

Alto	Authentication Credentials Captured
Description	An insecure authentication mechanism is in use. This allows an attacker on the network access to the userid and password of the authenticated user. For Basic Authentication, the attacker must merely monitor the network traffic until a Basic Authentication request is received, and then base64 decode the username and password. For Digest Authentication, the attacker has access to the username, and possibly also the password, if the hash (including a nonce) can be successfully cracked, or if a Man-In-The-Middle attack is mounted. The attacker eavesdrops on the network until an authentication has completed.
URL	http://localhost:8080/api/tutorials
Node Name	http://localhost:8080/api/tutorials
Método	GET
Atacar	
Evidence	
Other Info	[GET] [http://localhost:8080/api/tutorials] uses insecure authentication mechanism [Basic], revealing username [sa] and password [].
URL	http://localhost:8080/api/tutorials
Node Name	http://localhost:8080/api/tutorials (){{title,description,published}}
Método	POST
Atacar	
Evidence	
Other Info	[POST] [http://localhost:8080/api/tutorials] uses insecure authentication mechanism [Basic], revealing username [sa] and password [].
Instances	2
Solution	Use HTTPS, and use a secure authentication mechanism that does not transmit the userid or password in an un-encrypted fashion. In particular, avoid use of the Basic Authentication mechanism, since this trivial obfuscation mechanism is easily broken.
Reference	https://owasp.org/www-community/attacks/Brute_force_attack https://en.wikipedia.org/wiki/Digest_access_authentication

CWE Id	287
WASC Id	1
Plugin Id	10105
Médio	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://localhost:8080/h2-ui/
Node Name	http://localhost:8080/h2-ui/
Método	GET
Atacar	
Evidence	
Other Info	
URL	http://localhost:8080/h2-ui/login.do
Node Name	http://localhost:8080/h2-ui/login.do ()(driver,password,url,user)
Método	POST
Atacar	
Evidence	
Other Info	
URL	http://pbs.dei.isep.ipp.pt:8080/api/tutorials/1
Node Name	http://pbs.dei.isep.ipp.pt:8080/api/tutorials/1 ()({title,description,published})
Método	PUT
Atacar	
Evidence	
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/
CWE Id	693
WASC Id	15
Plugin Id	10038

Médio	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://localhost:8080/h2-ui/
Node Name	http://localhost:8080/h2-ui/
Método	GET
Atacar	
Evidence	
Other Info	
URL	http://localhost:8080/h2-ui/login.do
Node Name	http://localhost:8080/h2-ui/login.do ()(driver,password,url,user)
Método	POST
Atacar	
Evidence	
Other Info	
Instances	2
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020
Baixo	Application Error Disclosure
Description	This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.
URL	http://localhost:8080/api/tutorials/1
Node Name	http://localhost:8080/api/tutorials/1
Método	DELETE
Atacar	
Evidence	HTTP/1.1 500
Other Info	
URL	http://localhost:8080/api/tutorials?title=%27%20OR%20%271%27=%271
Node Name	http://localhost:8080/api/tutorials (title)
Método	GET

Atacar	
Evidence	HTTP/1.1 500
Other Info	
URL	http://localhost:8080/api/tutorials/abc_invalido
Node Name	http://localhost:8080/api/tutorials/abc_invalido
Método	GET
Atacar	
Evidence	HTTP/1.1 500
Other Info	
URL	http://localhost:8080/api/tutorials
Node Name	http://localhost:8080/api/tutorials ()({title,description,published})
Método	POST
Atacar	
Evidence	HTTP/1.1 500
Other Info	
Instances	4
Solution	Review the source code of this page. Implement custom error pages. Consider implementing a mechanism to provide a unique error reference/identifier to the client (browser) while logging the details on the server side and not exposing them to the user.
Reference	
CWE Id	550
WASC Id	13
Plugin Id	90022
Baixo	In Page Banner Information Leak
Description	The server returned a version banner string in the response content. Such information leaks may allow attackers to further target specific issues impacting the product and version in use.
URL	http://pbs.dei.isep.ipp.pt:8080/api/tutorials/1
Node Name	http://pbs.dei.isep.ipp.pt:8080/api/tutorials/1 ()({title,description,published})
Método	PUT
Atacar	
Evidence	Tomcat/9.0.10
Other Info	There is a chance that the highlight in the finding is on a value in the headers, versus the actual matched string in the response body.
Instances	Systemic
Solution	Configure the server to prevent such information leaks. For example: Under Tomcat this is done via the "server" directive and implementation of custom error pages. Under Apache this is done via the "ServerSignature" and "ServerTokens" directives.

Reference	https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/08-Testing_for_Error_Handling/
CWE Id	497
WASC Id	13
Plugin Id	10009
Baixo	Timestamp Disclosure - Unix
Description	A timestamp was disclosed by the application/web server. - Unix
URL	http://localhost:8080/api/tutorials
Node Name	http://localhost:8080/api/tutorials
Método	GET
Atacar	
Evidence	1773929290
Other Info	1773929290, which evaluates to: 2026-03-19 14:08:10.
Instances	1
Solution	Manually confirm that the timestamp data is not sensitive, and that the data cannot be aggregated to disclose exploitable patterns.
Reference	https://cwe.mitre.org/data/definitions/200.html
CWE Id	497
WASC Id	13
Plugin Id	10096
Baixo	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://localhost:8080/api/tutorials?title=Spring
Node Name	http://localhost:8080/api/tutorials (title)
Método	GET
Atacar	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://pbs.dei.isep.ipp.pt:8080/nifs-app/nifs
Node Name	http://pbs.dei.isep.ipp.pt:8080/nifs-app/nifs
Método	GET
Atacar	
Evidence	
Other	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages

Info	away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8080/api/tutorials
Node Name	http://localhost:8080/api/tutorials (){{title,description,published}}
Método	POST
Atacar	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security-Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	Authentication Request Identified
Description	The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.
URL	http://localhost:8080/h2-ui/login.do
Node Name	http://localhost:8080/h2-ui/login.do ()(driver,password,url,user)
Método	POST
Atacar	
Evidence	password
Other Info	userParam=user userValue=sa passwordParam=password referer=http://localhost:8080/h2-ui/login.jsp
Instances	1
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/
CWE Id	
WASC Id	
Plugin Id	10111

Informational	Content-Type Header Missing
Description	The Content-Type header was either missing or empty.
URL	http://localhost:8080/api/tutorials/1
Node	

Name	http://localhost:8080/api/tutorials/1
Método	DELETE
Atacar	
Evidence	
Other Info	
URL	http://localhost:8080/api/tutorials
Node Name	http://localhost:8080/api/tutorials
Método	OPTIONS
Atacar	
Evidence	
Other Info	
URL	http://localhost:8080/api/tutorials
Node Name	http://localhost:8080/api/tutorials (){title,description,published}
Método	POST
Atacar	
Evidence	
Other Info	
URL	http://localhost:8080/api/tutorials/1
Node Name	http://localhost:8080/api/tutorials/1 (){title,description,published}
Método	PUT
Atacar	
Evidence	
Other Info	
Instances	4
Solution	Ensure each page is setting the specific and appropriate content-type value for the content being delivered.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85)
CWE Id	345
WASC Id	12
Plugin Id	10019
Informational	Modern Web Application
Description	The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.
URL	http://localhost:8080/h2-ui/
Node Name	http://localhost:8080/h2-ui/
Método	GET

Atacar	
Evidence	<script type="text/javascript"> location.href = 'login.jsp?jsessionid=5ff8983dedf89eca9c70987f1a1e170e'; </script>
Other Info	No links have been found while there are scripts, which is an indication that this is a modern web application.
URL	http://localhost:8080/h2-ui/
Node Name	http://localhost:8080/h2-ui/
Método	GET
Atacar	
Evidence	<script type="text/javascript"> location.href = 'login.jsp?jsessionid=a3923bf8d0dd787f8bcee32bdfc59314'; </script>
Other Info	No links have been found while there are scripts, which is an indication that this is a modern web application.
URL	http://localhost:8080/h2-ui/login.do
Node Name	http://localhost:8080/h2-ui/login.do ()(driver,password,url,user)
Método	POST
Atacar	
Evidence	<script type="text/javascript"> location.href = 'login.jsp?jsessionid=8d761aa5a86a2b8b061ac6e2149579e3'; </script>
Other Info	No links have been found while there are scripts, which is an indication that this is a modern web application.
Instances	3
Solution	This is an informational alert and so no changes are required.
Reference	
CWE Id	
WASC Id	
Plugin Id	10109